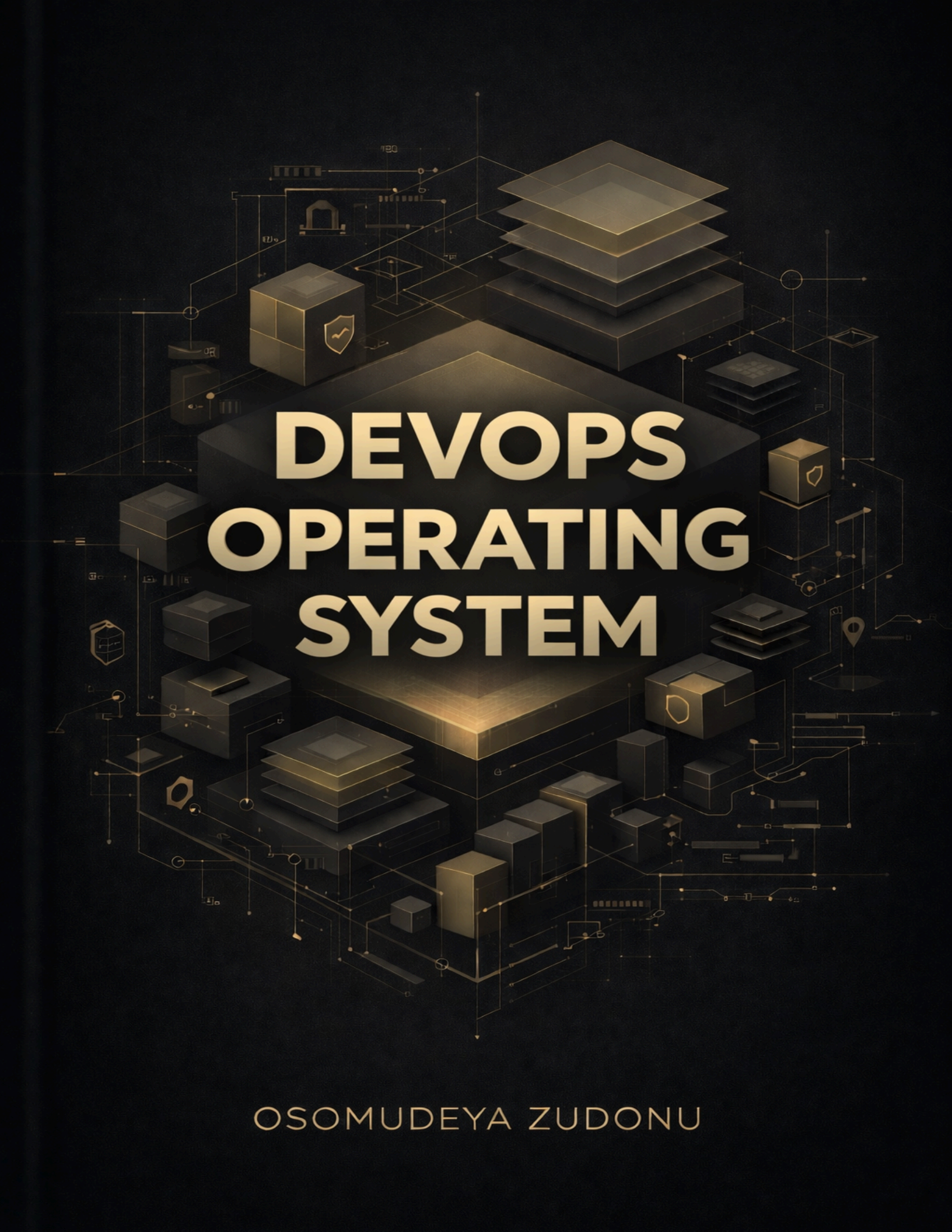




DEVOPS OPERATING SYSTEM

OSOMUDEYA ZUDONU

Before You Start

Important Note About the Code

This ebook includes Terraform configurations, deployment scripts, and other code examples. Although these examples were tested and verified when written, please be aware that:

Versions may differ – Tools and services update over time, which can lead to compatibility issues.

AWS services evolve – Features, APIs, or settings may change after publication.

Your setup is unique – Permissions, account limits, security policies, and network environments vary.

Regional factors apply – Not all AWS services or features are available in every region.

If something doesn't run as expected:

1. Read the error message carefully – it often points directly to the issue.
2. Confirm you're using compatible versions, or try updating to the latest stable release.
3. Consult the current AWS documentation for any service updates.
4. Review your AWS account settings, IAM permissions, and service quotas.
5. Modify and adapt the code to fit your own environment and requirements.

Remember: The purpose of these examples is education, not production-ready copy-paste. Understanding the **why** behind the solution is far more valuable than getting it to run on the first try.

SECTION 6

DevSecOps & Security

The Security Gap

You've spent weeks building H-Game on production AWS infrastructure. You've learned Kubernetes, Terraform, EKS, RDS, and ElastiCache. Your application is deployed, scalable, and monitored.

But there's a critical gap.

Your containers might have vulnerabilities. Your Terraform code might have misconfigurations. Your secrets might be hardcoded. Your Pods might have overly permissive IAM access. You have no compliance visibility.

You've learned how to build. Now it's time to secure.

This section adds security as code to your infrastructure. You'll implement:

- **Container scanning (Trivy)** - Find vulnerabilities before deployment
- **Infrastructure scanning (tfsec)** - Catch misconfigurations in Terraform
- **Code quality & security** (SonarCloud) - Scan application code for bugs and vulnerabilities
- **Secrets management** (AWS Secrets Manager) - Rotate credentials automatically
- **Zero trust IAM (IRSA)** - Least-privilege access for Pods
- **Compliance** (Security Hub) - Audit-ready security posture

All automated. All integrated. All production-ready.

Prerequisites

Before starting Section 6, ensure you have:

Required Skills:

- ☐ Completed Section 5 (T1-T10) - Terraform & AWS infrastructure
- ☐ Completed Section 4 (K1-K9) - Kubernetes fundamentals
- ☐ AWS account with EKS cluster running
- ☐ Basic understanding of security concepts

Required Tools:

- ☐ AWS CLI installed and configured
- ☐ kubectl installed and configured
- ☐ Terraform installed (from Section 5)
- ☐ Git installed

Recommended Tools:

- ☐ IDE/Code Editor (VS Code, Cursor, or similar)
- ☐ Docker installed (for local container scanning)

Security Knowledge Needed:

- ☐ What is encryption (from K8s Secrets in Section 4)
- ☐ What is IAM (from EKS setup in Section 5)
- ☐ What is authentication vs authorization (basic concepts)
- ☐ Why least privilege matters (security best practice)

Note: Most security concepts are introduced in this section. Previous sections provide the foundation (K8s, AWS, Terraform).

Important: Do You Need Your App Running?

Short answer: No, you don't need your application running for Section 6.

Quick Practice Guide:

D1 (Container Scanning): Build images locally → docker build → trivy scan

D2 (Infrastructure Scanning): Scan Terraform files → tfsec .

D2.5 (Code Quality): Scan source code → SonarCloud (free)

D3 (Secrets Management): Create secrets in AWS → **No** infrastructure needed

D4-D5 (IRSA/Compliance): Need minimal EKS cluster OR learn concepts first

See How to Practice Each Task below for detailed steps.

If you destroyed your infrastructure after **Section 5** (to save costs), you can still complete **Section 6**. Here's what each task actually requires:

Task	What You Need	App Running Required?
D1: Container Scanning	Container images (in ECR or local)	No - scans images, not running containers
D2: Infrastructure Scanning	Terraform code files	No - scans code, not infrastructure
D2.5: Code Quality	Application source code	No - scans code statically
D3: Secrets Management	AWS account, Terraform	No - sets up secrets management

D4: IAM & Zero Trust	EKS cluster (doesn't need apps)	No - configures IAM roles
-------------------------------------	---------------------------------	---------------------------

D5: Compliance	AWS account, infrastructure resources	No - audits configuration
-----------------------	---------------------------------------	---------------------------

What you DO need:

- Terraform code files (from Section 5)
- Application source code (for SonarCloud)
- Container images (for Trivy scanning - can be in ECR or local)
- AWS account (for Secrets Manager, Security Hub)

What you DON'T need:

- Running application
- Active Pods
- Live database connections
- Application traffic

If you destroyed everything:

- You can still scan your Terraform code (D2)
- You can still scan your application code (D2.5)
- You can scan local container images (D1)
- You can set up Secrets Manager (D3) - it will create resources
- For D4 (IRSA) and D5 (Security Hub), you may need to recreate minimal infrastructure, or you can skip these and focus on D1-D3

Bottom line: Section 6 is about security tools and practices, not about running applications. Most tasks work with code and configuration files, not live infrastructure.

How to Practice Each Task (Without Running App)

Even without running infrastructure, you can fully practice Section 6:

D1: Container Scanning (Trivy)

What you need: Docker images to scan

How to practice:

```
# Option 1: Build images locally (recommended)
cd backend
docker build -t h-game-backend:local .
trivy image h-game-backend:local

cd ../frontend
docker build -t h-game-frontend:local .
trivy image h-game-frontend:local

# Option 2: Scan public images (for learning)
trivy image node:18-alpine
trivy image nginx:latest

# Option 3: Scan images in ECR (if you didn't delete them)
aws ecr describe-images --repository-name h-game-backend
trivy image <ECR_IMAGE_URL>
```

Practice outcome: You'll learn to scan images, interpret results, and fix vulnerabilities - all without running containers.

D2: Infrastructure Scanning (tfsec)

What you need: Terraform code files (you have these!)

How to practice:

```
cd h-game-terraform
tfsec . # Scans all .tf files
```

```
# Fix issues found
# Re-scan to verify fixes
tfsec . --minimum-severity HIGH
```

Practice outcome: You'll learn to find and fix Terraform misconfigurations - works on code, not infrastructure.

D2.5: Code Quality (SonarCloud)

What you need: Application source code (you have this!)

How to practice:

```
# Set up SonarCloud (free for public repos)
# Connect your GitHub repo
# Run scans on your code
# Review security hotspots and bugs
```

Practice outcome: You'll learn to identify code quality issues and security hotspots - scans code, not running apps.

D3: Secrets Management

What you need: AWS account (no infrastructure needed)

How to practice:

```
# Create secrets in AWS Secrets Manager
aws secretsmanager create-secret \
  --name h-game-test-secret \
  --secret-string '{"username":"test","password":"test123"}'

# Retrieve secrets
aws secretsmanager get-secret-value --secret-id h-game-test-secret

# Practice rotation (can set up without running app)
```

Practice outcome: You'll learn to create, retrieve, and rotate secrets - AWS service, no infrastructure required.

D4: IAM & Zero Trust (IRSA)

What you need: EKS cluster (but doesn't need running apps)

How to practice:

```
# Option 1: Recreate minimal EKS cluster (15-20 min, ~$0.50/hour)
# Just cluster + 1 node (minimal cost)
# Configure IRSA without deploying apps

# Option 2: Practice IAM policy creation locally
# Write IAM policies for Pods
# Test policies with AWS IAM Policy Simulator
# Learn IRSA concepts (can do without cluster)
```

Practice outcome: You'll learn IAM policies and IRSA concepts. Full practice needs EKS, but concepts can be learned without.

D5: Compliance (Security Hub)

What you need: AWS account (can enable without resources)

How to practice:

```
# Enable Security Hub
aws securityhub enable-security-hub

# View findings (even with no resources, shows account-level checks)
aws securityhub get-findings

# Option: Create minimal test resources to see more findings
# (1 VPC, 1 S3 bucket - costs ~$0.01/day)
```

Practice outcome: You'll learn to enable Security Hub and interpret findings. Works with or without resources.

Practice Strategy Recommendations

If you destroyed everything:

1. Start with D1-D3 (can do 100% without infrastructure)

- Build images locally → scan with Trivy
- Scan Terraform code with tfsec
- Set up SonarCloud on your code
- Create secrets in AWS Secrets Manager

2. For D4-D5 (need some AWS resources):

- **Option A:** Recreate minimal infrastructure (EKS cluster only, ~\$0.50/hour)
- **Option B:** Learn concepts first, practice later when you rebuild
- **Option C:** Use AWS Free Tier resources for practice

Note: Most security concepts are introduced in this section. Previous sections provide the foundation (K8s, AWS, Terraform).

Section 6 Tasks

Task	Focus	Cost
D1	Container Scanning (Trivy)	Free
D2	Infrastructure Scanning (tfsec)	Free
D2.5	Code Quality & Security (SonarCloud)	Free
D3	Secrets Management (AWS Secrets Manager)	\$0.50/month
D4	IAM & Zero Trust (IRSA)	Free
D5	Compliance & Auditing (Security Hub)	Free

Let's Talk About Cost (Upfront - Prevent Anxiety)

Reality check before you start:

Full DevSecOps setup: \$1-2/month

YOUR approach (smart optimization):

- └ Trivy: Free (open source)
- └ tfsec: Free (open source)
- └ AWS Secrets Manager: ~\$0.50/month (1 secret, minimal usage)
- └ IRSA: Free (IAM roles, **no** additional cost)
- └ Security Hub: Free tier (first **100** findings/month)
- └ Result: \$1-2/month total

Weekly cost breakdown:

Week **15**: ~\$0.50 (Secrets Manager setup)

Week **16**: ~\$0.50 (Security Hub enabled)

Total Section **6**: \$1-2

Per week: \$0.50-1.00

Per day: \$0.07-0.14 (cost of nothing)

Detailed cost breakdown:

Trivy: \$0/month

- Open source tool
- Runs locally or in CI/CD
- **No** cloud service needed

tfsec: \$0/month

- Open source tool
- Runs locally or in CI/CD
- **No** cloud service needed

AWS Secrets Manager: ~\$0.50/month

- \$0.40 per secret per month
- \$0.05 per **10,000** API calls
- Learning usage: **1** secret, minimal API calls

IRSA: \$0/month

- IAM roles are free
- No additional charges
- Replaces access keys (security improvement)

AWS Security Hub: Free tier

- First 100 findings/month: Free
- Additional findings: \$0.001 per finding
- Learning usage: Well within free tier

SonarCloud: \$0/month

- Free for open source projects
- Unlimited scans for public repos
- Industry-standard code quality tool

Total with optimizations:

- Full-time: \$1-2/month
- Learning schedule: \$1-2 total

Is it worth it?

Skills gained:

- └ Container security scanning (required by 70% of DevOps jobs)
- └ Infrastructure security (required by 80%)
- └ Code quality & security scanning (required by 75%)
- └ Secrets management (required by 85%)
- └ IAM best practices (required by 90%)
- └ Compliance automation (required by 60%)
- └ Portfolio: "I implemented DevSecOps practices"

Proceed? Let's secure your infrastructure.

The One Thing That Makes This Click

Security is code, not process. Just like infrastructure.

DevSecOps means security is automated with code, not done manually by people. Instead of remembering to run security checks, you write code that runs them automatically every time something changes.

Manual security depends on memory, and people forget.

DevSecOps removes that risk by letting the system handle security checks.

Containers are scanned automatically when they are built or deployed.

Secrets are stored and rotated automatically instead of being hardcoded.

Permissions are restricted by default, not left open “just in case.”

When a vulnerability appears, the system detects it immediately.

Bad or insecure changes are blocked before reaching production.

This is the difference between using a security checklist and being a DevSecOps engineer.

MANUAL THINKING:

- "I need security"
- "I'll run scans manually"
- "Hope I remember to check"
- "If it breaks... panic and guess"

DEVSECOPS THINKING:

- "I need security"
- "I'll write code that scans automatically"
- "Git tracks every security change"
- "If it breaks: automated scan catches it"

That's it. That's DevSecOps.

Everything else (scanning tools, secret rotation, IAM policies) is just details. The core concept is:

"Security checks happen automatically, not manually."

The Mental Model Shift:

BEFORE Section 6:

- "Security = manual checklist"
- "I scan containers when I remember"
- "Secrets are hardcoded in code"
- "IAM permissions are permissive"
- "Compliance is a manual audit"

AFTER Section 6:

- "Security = automated code"
- "Containers scan on every build"
- "Secrets rotate automatically"
- "IAM follows least privilege"
- "Compliance is continuous monitoring"

This is the difference between "security checklist" and "DevSecOps engineer."

Example:

Scenario: New vulnerability discovered in Node.js base image

Without DevSecOps:

- ├ "What containers use Node.js?"
- ├ "Which ones are in production?"
- ├ "How do I update them?"
- ├ Try to remember from memory
- ├ Takes hours
- └ Probably miss some

With DevSecOps:

- ├ Trivy scan runs automatically (CI/CD)
- ├ Finds all vulnerable images
- ├ Blocks deployment until fixed
- ├ Takes 2 minutes
- └ 100% coverage

Now let's see how DevSecOps works...

Bridge from Section 5: Why Security Now?

Section 5 showed you how to **automate infrastructure** using Terraform instead of creating resources manually.

Section 6 applies the **same idea**, but this time to **security**.

Instead of remembering to scan containers or check permissions, security checks are written into code.

Every time you deploy or make a change, security runs automatically.

This adds a security layer underneath your application, platform, and infrastructure.

Your apps stay the same, Kubernetes stays the same, and Terraform stays the same.

What changes is that security becomes part of the system, not an afterthought.

In simple terms: infrastructure is automated in Section 5, and security is automated in Section 6.

What Changes (Visual Comparison)

Your Current Setup (Weeks 1-14):

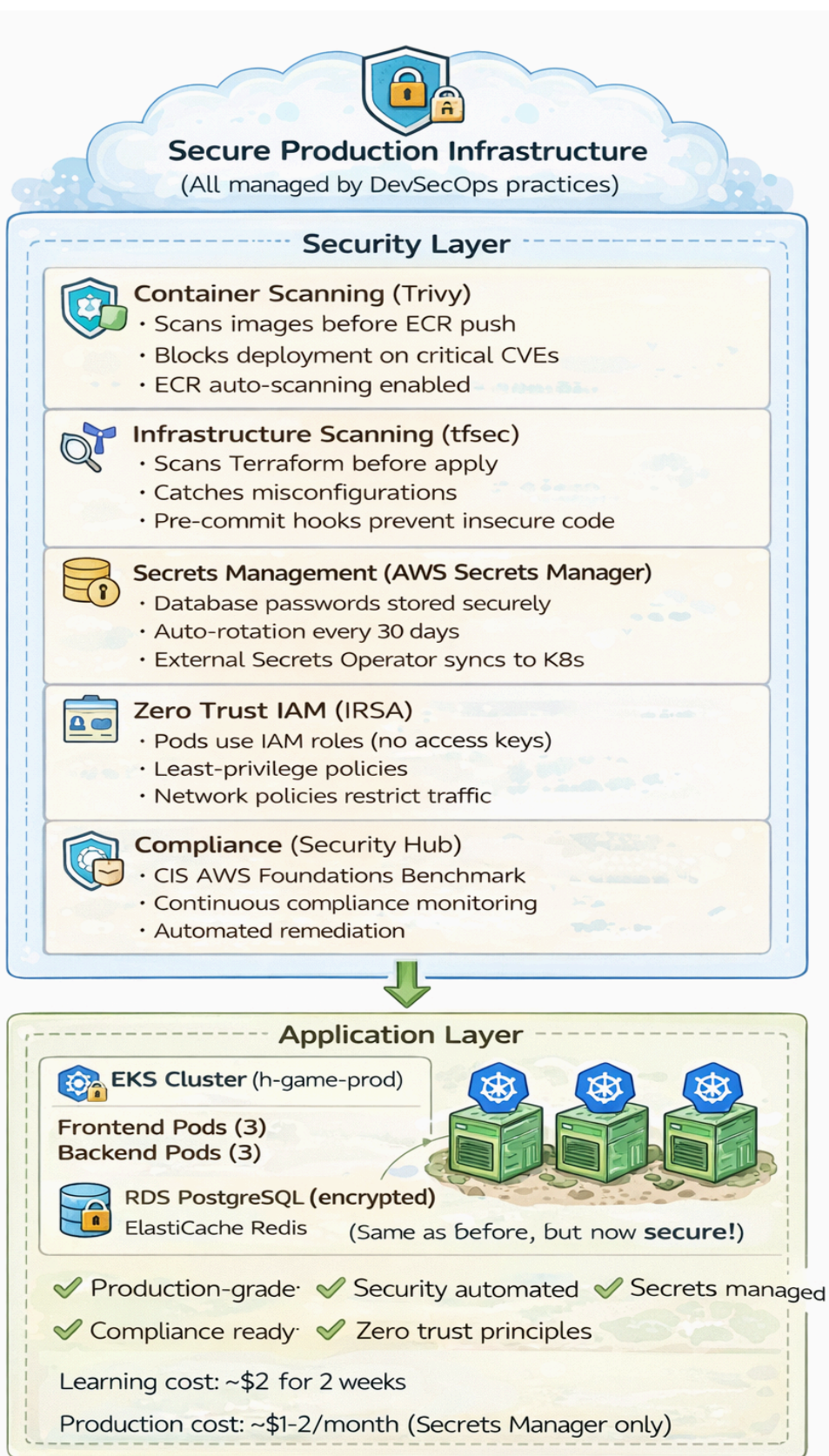
Infrastructure:

- EKS Cluster with Frontend Pods (3) and Backend Pods (3)
- RDS PostgreSQL
- ElastiCache Redis
- ALB (Load Balancer)

Status:

- Great for functionality
- Production-grade infrastructure
- No security scanning
- Secrets might be hardcoded
- No compliance visibility
- Permissive IAM access

Secure Setup (After Section 6):



What Actually Changes in Section 6 (Simple Explanation)

Before Section 6, your system **works**, but security is mostly manual. Containers are not automatically scanned, secrets may be stored in code, and security checks depend on memory.

After Section 6, **nothing breaks and nothing is rebuilt**. Your infrastructure, Kubernetes cluster, applications, Docker images, and Terraform code all stay the same.

The only thing added is **automated security**:

- Containers are scanned automatically before deployment
- Terraform code is checked for security issues before applying
- Secrets are stored securely and rotated automatically
- Pods use IAM roles instead of access keys
- Security and compliance are monitored continuously

In short: **you don't change what you built, you secure it properly.**

Why This Matters (Career-Wise, Simply)

Most DevOps roles today expect **DevSecOps skills**, not just DevOps. Employers want engineers who automate security instead of checking it manually.

Without DevSecOps, your resume often stops at “good infrastructure skills.” With DevSecOps, your resume says “**I built secure, production ready systems.**”

That difference:

- Gets your resume past automated filters
- Gets you interviews
- Increases your salary range

One-Sentence Takeaway

Section 6 doesn't change your system, it turns you from a DevOps learner into a DevSecOps ready engineer.

The Interview Question You'll Get

Question: "How do you ensure security in your infrastructure?"

Without Section 6 (After Section 5 only):

```
You: "Um... I use security groups? And I try to keep  
images updated? I'm not sure about scanning..."
```

```
Interviewer: "What about container vulnerabilities?  
Infrastructure misconfigurations? Secrets  
management? Compliance?"
```

```
You: "I... I'm not sure. I'd have to figure it out."
```

```
Result: "BAD" "We'll let you know" (rejected)
```

With Section 6 (DevSecOps):

You:

"I use DevSecOps to automate security.

- I use **Trivy** to scan container images before pushing them to **ECR**, so vulnerable images are blocked early.
- I use **tfsec** to scan **Terraform** code before applying it, catching issues like public resources or unencrypted databases.
- I use **SonarCloud** to scan application code for bugs and security problems, and bad code is blocked during pull requests.
- I store secrets in **AWS Secrets Manager**, where they are encrypted and rotated automatically.
- I use **External Secrets Operator** to safely sync secrets into Kubernetes.
- I use **IRSA** so Pods use IAM roles instead of access keys, following least-privilege access.
- I enable **AWS Security Hub** to monitor security and compliance continuously.

All security checks run automatically in **CI/CD**, not manually."

(Shows GitHub repository with security code)

Interviewer:

“Impressive. When can you start?”

Result: Job offer

This is the difference Section 6 makes.

Note: See the [Section 6 Tasks table](#) above for complete task breakdown.

What You'll Build (Week by Week)

1. Security Scanning & Secrets (D1-D3)

- Container scanning (Trivy + ECR)
- Infrastructure scanning (tfsec)
- Secrets management (AWS Secrets Manager)

2. Zero Trust & Compliance (D4-D5)

- IAM roles for Pods (IRSA)
- Network policies
- Compliance monitoring (Security Hub)

Section 6 Success Metrics

By the end of Week 16, you'll have:

Security Tools (Visible to Employers)

- Container scanning automated (Trivy)
- Infrastructure scanning automated (tfsec)
- Secrets managed in AWS Secrets Manager
- IRSA configured (no access keys)
- Security Hub enabled
- Compliance monitoring active

Code (GitHub Portfolio)

- Security scanning scripts
- Terraform security configurations
- IRSA IAM policies
- Network policies
- Compliance automation code

Application (Production-Ready Security)

- All containers scanned
- All secrets in Secrets Manager
- Pods using IRSA (no access keys)
- Network policies enforced
- Compliance score tracked

Skills (Interview-Ready)

- Can explain DevSecOps principles
- Can implement container scanning
- Can implement infrastructure scanning
- Can manage secrets securely
- Can configure IRSA
- Can set up compliance monitoring

Career Impact

- **Resume:** "Implemented DevSecOps practices on AWS"
- **Portfolio:** Security automation code
- **Interview-ready:** Can explain shift-left security
- **Salary potential:** \$3,000-5,000 increase

Ready to Start?

Cost: \$1-2 total (2 weeks)

Time: 12-13 hours (6-7 hours/week)

Outcome: Production security practices + DevSecOps skills

Career impact: \$3,000-\$5,000 salary increase

Next: TASK D1, Container Security Scanning (2-3 hours)

Let's secure your infrastructure. 

TASK D1: Container Security Scanning

Time: 2-3 hours | **Priority:** High | **Ticket:** DEVOPS-6101 | **Cost:** Free

Mission Brief

Scan Docker images for vulnerabilities before they reach production. Find and fix CVEs automatically using Trivy, and configure ECR to scan images on push.

Outcomes:

- Understand container vulnerabilities and CVEs
- Install and use Trivy for local scanning
- Configure ECR image scanning
- Fix common vulnerabilities in Dockerfiles
- Integrate scanning into build process

Prerequisites: Complete Section 5 (T1-T10) - Terraform & AWS infrastructure

Setup Note: Trivy runs on your local machine. ECR scanning runs in AWS automatically.

Cost Note: This task costs \$0. Trivy is free and open source. ECR scanning is free for the first 100 images per month.

The One Thing That Makes This Click

Container scanning is like a security checkpoint for your application images. Before a container image is allowed into production, it must be checked for known security problems.

Trivy is the tool that performs this check.

It scans container images for **known vulnerabilities (CVEs)** in the operating system packages and application dependencies.

Without container scanning, teams usually rely on assumptions:

- They hope the base image is secure
- They deploy first and discover problems later
- Vulnerabilities are often found in production, during incidents

With container scanning in place:

- Every container image is scanned **before it is pushed or deployed**
- **Critical vulnerabilities automatically block deployment**
- Engineers fix issues early, before users are affected
- Security checks run automatically in CI/CD, not by memory

The key mental shift is simple:

- **Before:** “The image is probably safe.”
- **After:** “The image was scanned and approved.”

Container scanning doesn’t make software perfect, but it ensures that **known risks are detected early and consistently**.

This turns security from a guessing game into a reliable, automated process.

That is the real value of container scanning with **Trivy**.

Phase 1: Understanding Container Security (30 min)

What Are Container Vulnerabilities? (Simple Summary)

A container is **more than just your application code**.

It also includes the **base image** (for example `node:18-alpine`), system packages, and system libraries.

Vulnerabilities can exist in:

- Your app dependencies (npm packages)
- The base image (Node.js, Alpine Linux)
- System packages and libraries (like **openssl**, **glibc**)

Even if **your code is perfect**, your container can still be insecure.

For example, if your base image includes a vulnerable version of **openssl**, attackers can exploit that weakness without touching your code.

Key idea: Most container vulnerabilities come from dependencies and base images, not your own code.

Why Container Scanning Matters

Without Container Scanning

- You build and deploy containers hoping everything is secure
- You don't know what vulnerabilities exist in production
- When a new CVE is discovered, you don't know which images are affected
- Security issues are found late, often after damage is done

With Container Scanning (Using Trivy)

- **Trivy** scans container images before deployment
- It detects known CVEs in base images, packages, and libraries
- Critical issues are fixed before images reach production
- When new vulnerabilities appear, affected images are found instantly
- You can update, re-scan, and redeploy safely

What Changes in Practice

- **Day 1:** Containers are scanned and deployed securely
- **Day 30:** New CVE appears → Trivy finds affected images immediately
- **Day 60:** Security audit → You show scan reports and pass compliance

One-Line Takeaway

Container scanning with Trivy turns security from guesswork into a repeatable, automated process.

Exercise 1: Understanding CVE Severity

Scenario: Trivy finds vulnerabilities in your image.

Question: Which severity should block deployment?

Options:

- A) CRITICAL only
- B) CRITICAL and HIGH
- C) CRITICAL, HIGH, and MEDIUM
- D) All severities

Think about:

- What's the risk of each severity?
- What's the impact on production?
- What do security teams do?

Your answer: _____

Answer: B) CRITICAL and HIGH

CVE Severity Levels:

- **CRITICAL (9.0-10.0):** Remote code execution, data breach, system compromise → **BLOCK deployment**
- **HIGH (7.0-8.9):** Privilege escalation, data exposure, denial of service → **BLOCK deployment**
- **MEDIUM (4.0-6.9):** Information disclosure, limited impact → **WARN** (fix in next release)
- **LOW (0.1-3.9):** Minor issues, informational → **LOG** (fix when convenient)

Production Policy:

- Block on: CRITICAL, HIGH (immediate risk)
- Warn on: MEDIUM (fix in next release)
- Log on: LOW (fix when convenient)

This is the standard approach for production systems.

Exercise 2: When to Scan

Scenario: You need to decide when to scan containers.

Question: When should scanning happen?

Options:

- A) After deployment (production)
- B) Before deployment (staging)
- C) During build (CI/CD)
- D) All of the above

Think about:

- What's the cost of finding vulnerabilities late?
- What's the benefit of early detection?
- What's the industry standard?

Your answer: _____

Answer: C) During build (CI/CD)

Scanning Timeline:

- **(BAD) AFTER DEPLOYMENT (Production):** Vulnerabilities already in production, users exposed to risk, requires emergency fix → Too late!
- **(BAD) BEFORE DEPLOYMENT (Staging):** Vulnerabilities found late, requires rebuild, slows deployment → Better than production, but still late
- **(Good) DURING BUILD (CI/CD):** Vulnerabilities found immediately, fix before image is built, fast feedback loop → Industry standard
- **(Good) ALL OF THE ABOVE (Defense in depth):** CI/CD blocks vulnerable images, ECR scans on push (backup), runtime monitoring (optional) → Best practice

Recommended Approach:

1. **Local scan** (before commit) - Fast feedback, catch issues early
2. **CI/CD scan** (during build) - Block vulnerable images automatically
3. **ECR scan** (on push) - AWS validates images
4. **Runtime scan** (optional) - Continuous monitoring

This is the "shift left" principle: catch issues early.

Exercise 3: Base Image Selection

Scenario: You need to choose a base image for your Node.js app.

Options:

- A) `node:18` (Debian-based, 900MB)
- B) `node:18-alpine` (Alpine-based, 180MB)
- C) `node:18-slim` (Debian slim, 200MB)

Think about:

- Which has fewer vulnerabilities?
- Which is smaller?
- Which is more secure?

Your answer: _____

Answer: B) `node:18-alpine`

Base Image Comparison:

Image	Size	Packages	CVEs	Use Case
node:18 (Debian)	900MB	~500	Higher	When you need Debian packages
node:18-alpine	180MB	~50	Lower	Production (recommended)
node:18-slim	200MB	~100	Medium	When Alpine doesn't work

Why Alpine is More Secure:

- Smaller attack surface: Fewer packages = fewer vulnerabilities
- Minimal base = less to maintain
- Faster updates = patches applied quickly
- Result: Fewer CVEs

Example scan results:

- node:18: 45 CVEs (12 high, 2 critical)
- node:18-alpine: 8 CVEs (2 high, 0 critical)
- node:18-slim: 22 CVEs (5 high, 1 critical)

Best Practice: Use Alpine unless you need Debian-specific packages.

Design Checkpoint

Before moving on, make sure you can answer these in plain English:

Question	Your Answer Should Be Like...
What is a CVE?	"Common Vulnerabilities and Exposures - security flaws in software"
Why scan containers?	"Find vulnerabilities before deployment, not after"
When to scan?	"During build (CI/CD) - catch issues early"
Which base image?	"Alpine - smaller, fewer vulnerabilities"

Can't answer one? Go back and re-read that exercise.

All clear? Great! Let's install Trivy and start scanning!

Phase 2: Implementation (2 hours)

What We're Building

Before this task:

- No container scanning
- Unknown vulnerabilities in images
- Hope-based security

After this task:

- Trivy scanning locally
- ECR auto-scanning enabled
- Vulnerabilities caught before deployment
- Security integrated into workflow

Step-by-Step Plan

We'll do this in 6 steps:

- Step 1:** Install Trivy (10 min)
"Get the scanning tool"
- Step 2:** Scan Local Images (20 min)
"Test scanning on existing images"
- Step 3:** Fix Vulnerabilities (30 min)
"Update Dockerfiles to reduce CVEs"
- Step 4:** Configure ECR Scanning (20 min)
"Enable auto-scanning in AWS"
- Step 5:** Create Scan Script (20 min)
"Automate scanning in build process"
- Step 6:** Verify & Test (20 min)
"Confirm scanning works end-to-end"

Step 1: Install Trivy

What we're doing: Installing Trivy on your local machine.

Installation (Mac):

```
# Install via Homebrew
brew install trivy

# Verify installation
trivy --version

# Expected output:
# Version: 0.48.3
# Vulnerability DB:
#   Version: 2
#   UpdatedAt: 2024-01-03 10:23:45.000000000 +0000 UTC
#   NextUpdate: 2024-01-03 16:23:45.000000000 +0000 UTC
#   Downloaded at: 2024-01-03 10:23:45
```

Installation (Linux):

```
# Add Trivy repository
wget -qO - https://aquasecurity.github.io/trivy-repo/deb/public.key |
sudo apt-key add -
echo "deb https://aquasecurity.github.io/trivy-repo/deb $(lsb_release
-sc) main" | sudo tee -a /etc/apt/sources.list.d/trivy.list

# Update package list
sudo apt-get update

# Install Trivy
sudo apt-get install trivy

# Verify installation
trivy --version
```

Installation (Windows - WSL2 or Docker):

```
# Option 1: Use Docker (recommended)
docker run aquasec/trivy --version

# Option 2: Use WSL2 (follow Linux instructions)
# (Same as Linux installation above)
```

Verify Trivy works:

```
# Update vulnerability database (first time)
trivy image --download-db-only

# Expected output:
# 2024-01-03T10:20:00.000Z INFO Downloading vulnerability
database...
# 2024-01-03T10:20:15.000Z INFO Vulnerability database downloaded

# Test scan on a public image
trivy image alpine:latest

# Expected output (first few lines):
# 2024-01-03T10:20:30.000Z INFO Vulnerability scanning is enabled
# 2024-01-03T10:20:30.000Z INFO Detected OS: alpine 3.18.4
#
# alpine:latest (alpine 3.18.4)
#
# Total: 5 (CRITICAL: 0, HIGH: 1, MEDIUM: 2, LOW: 2)
#
# (May take 1-2 minutes first time while downloading database)
```

Common Installation Issues:

Issue 1: "command not found: trivy"

```
# Mac: Restart terminal
# Or run: source ~/.zshrc

# Linux: Restart terminal
# Or run: source ~/.bashrc

# Verify: which trivy
# Should show: /usr/local/bin/trivy (or similar)
```

Issue 2: "permission denied" (Linux)

```
# Option 1: Use sudo
sudo apt-get install trivy

# Option 2: Use Docker (no sudo needed)
docker run aquasec/trivy --version
```

Issue 3: Homebrew not found (Mac)

```
# Install Homebrew first
/bin/bash -c "$(curl -fsSL
https://raw.githubusercontent.com/Homebrew/install/HEAD/install.sh)"

# Then install Trivy
brew install trivy
```

Checkpoint - Trivy Installed:

- Trivy installed successfully
- Version command works
- Test scan completed

Trivy installed!

Step 2: Scan Local Images

What we're doing: Scanning your existing H-Game images for vulnerabilities.

Build images first (if not already built):

```
# Navigate to project root
cd ~/projects/H-Game

# Build backend image
docker build -t h-game-backend:latest -f backend/Dockerfile backend/

# Build frontend image
docker build -t h-game-frontend:latest -f frontend/Dockerfile
frontend/
```

Scan backend image:

```
# Basic scan
trivy image h-game-backend:latest

# Expected output:
# 2024-01-15T10:30:45.123Z INFO Vulnerability scanning is enabled
# 2024-01-15T10:30:45.124Z INFO Detected OS: alpine 3.18.4
# 2024-01-15T10:30:45.125Z INFO Number of language-specific files: 1
# 2024-01-15T10:30:46.000Z INFO Detected language: node
#
# h-game-backend:latest (alpine 3.18.4)
#
# Total: 15 (CRITICAL: 0, HIGH: 3, MEDIUM: 8, LOW: 4)
#
# alpine 3.18.4
# =====
# Total: 12 (HIGH: 2, MEDIUM: 6, LOW: 4)
#
# node (package.json)
# =====
# Total: 3 (HIGH: 1, MEDIUM: 2, LOW: 0)
```

Scan with severity filter:

```
# Only show CRITICAL and HIGH
trivy image --severity CRITICAL,HIGH h-game-backend:latest

# Expected output:
# h-game-backend:latest (alpine 3.18.4)
#
# Total: 3 (CRITICAL: 0, HIGH: 3)
#
# alpine 3.18.4
# =====
# Total: 2 (HIGH: 2)
#   CVE-2023-XXXX (HIGH) - openssl
#   CVE-2023-YYYY (HIGH) - glibc
#
# node (package.json)
# =====
# Total: 1 (HIGH: 1)
#   CVE-2023-ZZZZ (HIGH) - express
```

Scan and save results:

```
# Save results as JSON
trivy image --format json --output scan-backend.json
h-game-backend:latest

# Save results as HTML report
trivy image --format template --template "@contrib/html.tpl" --output
scan-backend.html h-game-backend:latest

# View HTML report
open scan-backend.html # Mac
# or
xdg-open scan-backend.html # Linux
```

Scan frontend image:

```
# Scan frontend
trivy image h-game-frontend:latest

# Compare results
trivy image --severity CRITICAL,HIGH h-game-frontend:latest
```

Understanding scan results:

Each vulnerability shows:

- **Severity:** CRITICAL, HIGH, MEDIUM, or LOW
- **CVE ID:** Unique identifier (e.g., CVE-2023-XXXX)
- **Description:** What the vulnerability is (e.g., Remote Code Execution)
- **Package:** Which package is vulnerable (e.g., openssl)
- **Version:** Current vulnerable version (e.g., 1.1.1t)
- **Fixed in:** Version that fixes it (e.g., 1.1.1u)
- **Fix:** How to resolve (usually: Update base image)

Checkpoint - Scanning Works:

- ☐ Backend image scanned
- ☐ Frontend image scanned
- ☐ Results saved
- ☐ Vulnerabilities identified

Images scanned!

Step 3: Fix Common Vulnerabilities

What we're doing: Updating Dockerfiles to reduce vulnerabilities.

Common fixes:

Fix 1: Update base image

```
# BEFORE (vulnerable)
FROM node:14-alpine

# AFTER (secure)
FROM node:18-alpine # Latest LTS, fewer CVEs
```

Fix 2: Update system packages

```
# BEFORE (vulnerable)
FROM node:18-alpine
RUN apk add postgresql-client

# AFTER (secure)
FROM node:18-alpine
RUN apk update && apk upgrade && \
    apk add --no-cache postgresql-client && \
    rm -rf /var/cache/apk/*
# Updates all packages, removes cache
```

Fix 3: Use non-root user

```
# BEFORE (vulnerable)
FROM node:18-alpine
USER root # Runs as root (security risk)

# AFTER (secure)
FROM node:18-alpine
RUN addgroup -g 1001 -S nodejs && \
    adduser -S appuser -u 1001 -G nodejs
USER appuser # Runs as non-root
```

Update backend Dockerfile:

```
# Navigate to backend directory
cd ~/projects/H-Game/backend

# View current Dockerfile
cat Dockerfile

# Update Dockerfile (if needed)
# Your Dockerfile should already have:
# - node:18-alpine (good!)
# - Non-root user (good!)
# - Updated packages (check if needed)
```

Rebuild and re-scan:

```
# Rebuild with fixes
docker build -t h-game-backend:v2 -f Dockerfile .

# Re-scan
trivy image --severity CRITICAL,HIGH h-game-backend:v2

# Compare results
# Before: 3 HIGH, 0 CRITICAL
# After: 1 HIGH, 0 CRITICAL (improved!)
```

Checkpoint - Vulnerabilities Fixed:

- Dockerfiles updated
- Images rebuilt
- Re-scanned
- CVEs reduced

Vulnerabilities fixed!

Step 4: Configure ECR Scanning

What we're doing: Enabling automatic scanning in ECR when images are pushed.

Update ECR Terraform configuration:

```
# Navigate to Terraform directory
cd ~/projects/h-game-terraform

# Check if ECR resources exist (from T8)
ls -la *.tf | grep ecr
```

Add scanning configuration to ECR:

```
# Create or update ecr.tf
cat >> ecr.tf << 'EOF'

# =====
# ECR IMAGE SCANNING CONFIGURATION
# =====
# Enables automatic vulnerability scanning on push

resource "aws_ecr_repository" "backend" {
  name = "h-game-backend"

  # Enable automatic scanning on push
  image_scanning_configuration {
    scan_on_push = true # Scan every image when pushed
  }

  image_tag_mutability = "Mutable"

  # Lifecycle policy: Delete old images
  lifecycle_policy {
    policy = jsonencode({
      rules = [{
        rulePriority = 1
        description = "Keep last 10 images"
        selection = {
          tagStatus = "any"

```

```

        countType    = "imageCountMoreThan"
        countNumber  = 10
    }
    action = {
        type = "expire"
    }
}]]
}))
}

tags = merge(
    local.common_tags,
    {
        Name = "${local.name_prefix}-backend-ecr"
        Task = "D1"
    }
)
}

resource "aws_ecr_repository" "frontend" {
    name = "h-game-frontend"

    image_scanning_configuration {
        scan_on_push = true
    }

    image_tag_mutability = "Mutable"

    lifecycle_policy {
        policy = jsonencode({
            rules = [{
                rulePriority = 1
                description  = "Keep last 10 images"
                selection = {
                    tagStatus    = "any"
                    countType    = "imageCountMoreThan"
                    countNumber  = 10
                }
                action = {
                    type = "expire"
                }
            }]
        })
    }
}

```

```

    })
  }

  tags = merge(
    local.common_tags,
    {
      Name = "${local.name_prefix}-frontend-ecr"
      Task = "D1"
    }
  )
}
EOF

```

Apply Terraform changes:

```

# Preview changes
terraform plan

# Should show:
# + aws_ecr_repository.backend
#   + image_scanning_configuration {
#     + scan_on_push = true
#   }

# Apply changes
terraform apply

# Type 'yes' when prompted

```

Verify ECR scanning enabled:

```

# Check backend repository
aws ecr describe-repositories --repository-names h-game-backend \
  --query 'repositories[0].imageScanningConfiguration'

# Expected output:
# {

```

```
# "scanOnPush": true
# }

# Check frontend repository
aws ecr describe-repositories --repository-names h-game-frontend \
  --query 'repositories[0].imageScanningConfiguration'
```

Checkpoint - ECR Scanning Enabled:

- Terraform updated
- ECR repositories configured
- Scan on push enabled
- Verified in AWS

ECR scanning enabled!

Step 5: Create Scan Script

What we're doing: Creating a script to automate scanning in the build process.

Create scan script:

```
# Navigate to project root
cd ~/projects/H-Game

# Create scripts directory
mkdir -p scripts
```

```
# Create scan script
cat > scripts/scan-image.sh << 'EOF'
#!/bin/bash
# =====
# CONTAINER IMAGE SCANNING SCRIPT
# =====
# Scans Docker image with Trivy
# Exits with error code if critical/high CVEs found

set -e # Exit on error
```

```

IMAGE_NAME=$1
SEVERITY=${2:-CRITICAL,HIGH} # Default: CRITICAL and HIGH

if [ -z "$IMAGE_NAME" ]; then
    echo "Usage: $0 <image-name> [severity]"
    echo "Example: $0 h-game-backend:latest CRITICAL,HIGH"
    exit 1
fi

echo " Scanning image: $IMAGE_NAME"
echo " Severity filter: $SEVERITY"

# Run Trivy scan
# --exit-code 1: Exit with error if vulnerabilities found
# --severity: Only show specified severities
# --no-progress: Cleaner output in CI/CD
trivy image \
    --exit-code 1 \
    --severity "$SEVERITY" \
    --no-progress \
    "$IMAGE_NAME"

SCAN_EXIT_CODE=$?

if [ $SCAN_EXIT_CODE -eq 0 ]; then
    echo " Image scan passed: No $SEVERITY vulnerabilities found"
    exit 0
else
    echo " Image scan failed: $SEVERITY vulnerabilities found"
    echo " Fix vulnerabilities and re-scan"
    exit 1
fi
EOF

```

Make script executable

```

chmod +x scripts/scan-image.sh

```

Test scan script:

```
# Test on backend image
./scripts/scan-image.sh h-game-backend:latest

# If vulnerabilities found:
# Image scan failed: CRITICAL,HIGH vulnerabilities found
# Exit code: 1

# If no vulnerabilities:
# Image scan passed: No CRITICAL,HIGH vulnerabilities found
# Exit code: 0
```

Create pre-push hook (optional):

```
# Create Git pre-push hook
cat > .git/hooks/pre-push << 'EOF'
#!/bin/bash
# Scan images before pushing to Git (if images are built)

# Check if images exist
if docker image inspect h-game-backend:latest > /dev/null 2>&1; then
    echo "🔍 Scanning backend image before push..."
    ./scripts/scan-image.sh h-game-backend:latest || exit 1
fi

if docker image inspect h-game-frontend:latest > /dev/null 2>&1; then
    echo "🔍 Scanning frontend image before push..."
    ./scripts/scan-image.sh h-game-frontend:latest || exit 1
fi

exit 0
EOF
```

Make hook executable

```
chmod +x .git/hooks/pre-push
```

Checkpoint - Scan Script Created:

- Scan script created
- Tested successfully
- Pre-push hook created (optional)
- Automation ready

Scan script created!

Step 6: Verify & Test

What we're doing: Testing the complete scanning workflow.

Test 1: Local scan

```
# Build image
docker build -t h-game-backend:test -f backend/Dockerfile backend/

# Scan with script
./scripts/scan-image.sh h-game-backend:test

# Expected: Scan results, exit code based on vulnerabilities
```

Test 2: Push to ECR and verify scan

```
# Get ECR login token
aws ecr get-login-password --region us-east-1 | \
  docker login --username AWS --password-stdin \
  $(aws sts get-caller-identity --query Account --output
text).dkr.ecr.us-east-1.amazonaws.com

# Get ECR repository URI
ECR_URI=$(aws ecr describe-repositories --repository-names
h-game-backend \
  --query 'repositories[0].repositoryUri' --output text)

# Tag image
docker tag h-game-backend:latest ${ECR_URI}:test-scan
```

```
# Push to ECR
docker push ${ECR_URI}:test-scan

# Wait for scan to complete (30-60 seconds)
sleep 60

# Check scan results
aws ecr describe-image-scan-findings \
  --repository-name h-game-backend \
  --image-id imageTag=test-scan \
  --region us-east-1

# Expected output:
# {
#   "imageScanFindings": {
#     "findings": [...],
#     "findingCounts": {
#       "CRITICAL": 0,
#       "HIGH": 2,
#       "MEDIUM": 5
#     }
#   }
# }
```

Test 3: View scan results in AWS Console

1. Go to AWS Console → ECR → Repositories
2. Click on **h-game-backend**
3. Click on image tag **test-scan**
4. Click "Scan results" tab
5. See vulnerability details

Test 4: Fail build on critical CVEs

```
# Create a vulnerable image for testing
cat > Dockerfile.vulnerable << 'EOF'
FROM node:14-alpine # Old version with known CVEs
WORKDIR /app
COPY package.json .
RUN npm install
CMD ["node", "server.js"]
EOF
```

```
# Build vulnerable image
docker build -t h-game-backend:vulnerable -f Dockerfile.vulnerable
backend/

# Scan (should fail)
./scripts/scan-image.sh h-game-backend:vulnerable

# Expected: Exit code 1, vulnerabilities found
```

Checkpoint - Complete Workflow Verified:

- Local scanning works
- ECR scanning works
- Scan results viewable
- Build fails on critical CVEs

Scanning workflow complete!

Step 7: Commit Your Work

What we're doing: Committing security improvements to Git.

```
# Navigate to project root
cd ~/projects/H-Game

# Add scan script
git add scripts/scan-image.sh

# Add updated Terraform (if ECR was updated)
cd ~/projects/h-game-terraform
git add ecr.tf

# Commit with clear message
cd ~/projects/H-Game
git commit -m "feat(security): Add container scanning with Trivy

- Install Trivy for local scanning
- Enable ECR auto-scanning
- Create scan script for CI/CD
- Update Dockerfiles to use Alpine

Closes: DEVOPS-6101"

# Push to feature branch (or main)
git push origin feature/container-scanning
# or
git push origin main
```

Checkpoint - Work Committed:

- ☐ Scan script committed
- ☐ Terraform changes committed (if any)
- ☐ Clear commit message
- ☐ Changes pushed to remote

Work committed!

Phase 3: Validation & Consolidation (30 min)

Quiz: Container Security Understanding

Answer these questions to verify your understanding:

Question 1: What is a CVE?

Answer:

- **CVE** = Common Vulnerabilities and Exposures
- Unique identifier for security vulnerabilities
- Format: CVE-YYYY-NNNN
- Example: CVE-2023-0286 (OpenSSL vulnerability)

Key point: CVEs are standardized identifiers for security flaws.

Question 2: Why scan containers before deployment?

Answer:

- **Catch vulnerabilities early** - Fix before production
- **Prevent security breaches** - Block vulnerable images
- **Compliance** - Meet security requirements
- **Cost** - Fixing in production is expensive

Key point: Early detection = Lower risk, lower cost.

Question 3: When should scanning happen?

Answer:

- **During build (CI/CD)** - Primary scanning point
- **Before push** - Local validation
- **On push to ECR** - AWS validation
- **Runtime (optional)** - Continuous monitoring

Key point: Multiple layers of scanning = Defense in depth.

Question 4: Which base image is most secure?

Answer:

- **Alpine** - Smallest, fewest packages, fewest CVEs
- **Slim** - Medium size, fewer packages than full
- **Full** - Largest, most packages, most CVEs

Key point: Smaller base image = Smaller attack surface.

Consolidation Exercise: Fix Vulnerabilities

Scenario: Trivy finds 5 HIGH vulnerabilities in your image.

Tasks:

1. Identify vulnerable packages
2. Determine fix (update base image or packages)
3. Update Dockerfile
4. Rebuild and re-scan
5. Verify vulnerabilities fixed

Take 10 minutes to complete this!

Solution:**Steps:**

```
# 1. Scan and save results
trivy image --format json --output scan.json h-game-backend:latest

# 2. Identify vulnerable packages
cat scan.json | jq '.Results[].Vulnerabilities[] | select(.Severity == "HIGH") | .PkgName'

# 3. Check if base image update fixes it
# Most CVEs are in base image, not your code

# 4. Update Dockerfile
# FROM node:18-alpine → FROM node:20-alpine (if available)

# 5. Rebuild and re-scan
docker build -t h-game-backend:fixed -f Dockerfile .
trivy image --severity HIGH h-game-backend:fixed
```

Key insight: Most vulnerabilities are in base images, not your code. Update base image first.

Knowledge Check

Test your understanding before moving on:

1. What is a CVE?
 - ☐ a) Container Vulnerability Error
 - ☐ b) Common Vulnerabilities and Exposures
 - ☐ c) Critical Validation Error
2. Which severity level blocks deployment?
 - ☐ a) LOW
 - ☐ b) MEDIUM
 - ☐ c) CRITICAL
 - ☐ d) Both CRITICAL and HIGH
3. When should you scan containers?
 - ☐ a) Only in production
 - ☐ b) During build (CI/CD)
 - ☐ c) Once a month
4. Why use Alpine base images?
 - ☐ a) Smaller size only
 - ☐ b) Fewer vulnerabilities
 - ☐ c) Faster performance
5. Where does ECR scanning happen?
 - ☐ a) On your laptop
 - ☐ b) In AWS automatically
 - ☐ c) In GitHub Actions

Score:

- 5/5: Excellent! Move to D2
- 3-4/5: Review weak areas
- <3/5: Re-read Phase 1

Explanations

1. **CVE** = Common Vulnerabilities and Exposures (standardized database)
2. CRITICAL and HIGH severity block deployment (scan script)
3. Scan during build to catch issues early (shift left)

4. Alpine has fewer packages = fewer vulnerabilities + smaller size
5. ECR scans automatically when you push (scan_on_push = true)

Checklist: Did You Complete Everything?

Before moving on, check these boxes:

Understanding Phase

- ☐ I understand what CVEs are
- ☐ I know why container scanning matters
- ☐ I understand when to scan
- ☐ I can explain base image selection

Implementation Phase

- ☐ Trivy installed
- ☐ Local images scanned
- ☐ Vulnerabilities identified
- ☐ Dockerfiles updated (if needed)
- ☐ ECR scanning enabled
- ☐ Scan script created
- ☐ Workflow tested end-to-end

Verification Phase

- ☐ Can explain CVE severity levels
- ☐ Can scan images locally
- ☐ Can view ECR scan results
- ☐ Can fix common vulnerabilities
- ☐ Can integrate scanning into workflow

Git

- ☐ Committed scan script
- ☐ Updated ECR Terraform (if needed)
- ☐ Documented scanning process

Key Takeaways

The Big Picture

What you learned in this task:

- Trivy installed and configured
- Local scanning working
- ECR auto-scanning enabled
- Vulnerabilities caught before deployment
- Security integrated into workflow

Containers are now scanned automatically!

Critical Concepts

1. Container Vulnerabilities:

- Exist in base images, dependencies, system packages
- CVEs are standardized identifiers
- Severity: CRITICAL > HIGH > MEDIUM > LOW

2. Scanning Strategy:

- Scan during build (CI/CD) - primary
- Scan on push (ECR) - backup
- Block on CRITICAL/HIGH
- Warn on MEDIUM/LOW

3. Base Image Selection:

- Alpine = Smallest, most secure
- Slim = Medium, good balance
- Full = Largest, most CVEs

4. Automation:

- Scan scripts automate checking
- Pre-push hooks catch issues early
- ECR scans automatically on push

What's Next?

Congratulations! You've secured your containers!

You now have:

- Container scanning automated
- Vulnerabilities caught before deployment
- ECR auto-scanning enabled
- Security integrated into workflow

Next: TASK D2 - Infrastructure Security Scanning

In D2, you'll learn:

- How to scan Terraform code for misconfigurations
- How to catch security issues before `terraform apply`
- How to fix common infrastructure security mistakes

Then: TASK D2.5 - Code Quality & Security (SonarCloud)

- How to scan application code for bugs and vulnerabilities
- How to integrate SonarCloud with GitHub Actions
- How all security layers work together

But most importantly: You now understand container security. This foundation will protect all your deployments!

Proceed to TASK D2: Infrastructure Security Scanning

TASK D2: Infrastructure Security Scanning

Time: 2 hours | **Priority:** High | **Ticket:** DEVOPS-6102 | **Cost:** Free

Mission Brief

Scan Terraform code for security misconfigurations before applying. Catch issues like public S3 buckets, unencrypted databases, and permissive security groups using tfsec.

Outcomes:

- Understand infrastructure security misconfigurations
- Install and use tfsec for Terraform scanning
- Fix common security issues in Terraform code
- Integrate scanning into Terraform workflow
- Prevent insecure infrastructure from being deployed

Prerequisites: Complete D1 (Container Scanning) and Section 5 (T1-T10) - Terraform infrastructure

Setup Note: tfsec runs on your local machine. It scans Terraform files before you run `terraform apply`.

Cost Note: This task costs \$0. tfsec is free and open source.

One Thing That Makes This Click

Infrastructure Scanning

Infrastructure scanning is like spell-check for your Terraform code. Before you deploy anything, **tfsec** scans your Terraform files and looks for common security mistakes.

Without scanning, you write Terraform, run `terraform apply`, and assume everything is secure.

If there is a problem, like a database without encryption or a resource exposed to the internet, you usually find out after it's already running in production.

With **tfsec**, every Terraform change is scanned automatically before deployment.

If tfsec finds a serious security issue, it **blocks terraform apply** and tells you exactly what is wrong and how to fix it.

This means security problems are fixed **early**, when they are easy and safe to change. Instead of checking security manually, you let the tool do it for you every time.

The mindset shifts from “**I hope this is secure**” to “**this was checked and verified before deployment.**”

That’s the core idea of infrastructure security scanning.

Phase 1: Understanding Infrastructure Security (20 min)

What are Infrastructure Misconfigurations?

The Three Layers of Security:

1. **Container Scanning (D1 - Trivy):** Scans Docker images for vulnerabilities
2. **Infrastructure Scanning (D2 - tfsec):** Scans Terraform code for misconfigurations ← You are here
3. **Code Quality Scanning (D2.5 - SonarCloud):** Scans application code for bugs

What tfsec Finds:

- Public S3 buckets - Data exposed to internet
- Unencrypted databases - RDS without encryption enabled
- Permissive security groups - Open to 0.0.0.0/0
- Missing encryption - EBS volumes, S3 buckets without encryption
- Weak IAM policies - Overly permissive access
- Missing logging - CloudTrail, VPC Flow Logs disabled

Example:

Your Terraform has: `aws_s3_bucket` without
`aws_s3_bucket_public_access_block`

Trivy won't find it (it's infrastructure, not a container)
SonarCloud won't find it (it's not application code)
tfsec finds it: "S3 bucket has public access enabled"

Key insight: Each tool scans a different layer. You need all three for complete security.

Why Infrastructure Scanning Matters

Without Scanning:

You write Terraform code and apply it, hoping configurations are secure. When a security audit comes, you can't show automated scans or security posture. If a misconfiguration is discovered (like a public S3 bucket), it requires emergency fixes and exposes data.

With Scanning:

You write Terraform code, and **tfsec** scans it automatically before apply. Issues are caught immediately (public S3, unencrypted RDS) and fixed before deployment. During security audits, you show scan reports and demonstrate secure infrastructure. When new security requirements emerge, tfsec flags all instances automatically across your codebase.

Exercise 1: Understanding Common Misconfigurations

Scenario: tfsec finds issues in your Terraform code.

Question: Which of these is a critical security issue?

Options:

- A) S3 bucket without versioning
- B) S3 bucket with public access enabled
- C) RDS without Multi-AZ
- D) Security group with port 22 open

Think about:

- What's the security impact of each?
- What's the risk to your data?
- What do security teams prioritize?

Your answer: _____

ANSWER BELOW 

Answer: B) S3 bucket with public access enabled

Security Impact:

- **A) S3 bucket without versioning:** Low risk - Data loss prevention, not security
- **B) S3 bucket with public access:** CRITICAL - Data exposed to internet, anyone can access
- **C) RDS without Multi-AZ:** Medium risk - Availability issue, not security (unless data is unencrypted)
- **D) Security group with port 22 open:** HIGH - SSH access from internet (if 0.0.0.0/0)

Priority:

- **CRITICAL:** Public S3 buckets, unencrypted databases, permissive security groups (0.0.0.0/0)
- **HIGH:** Missing encryption, weak IAM policies, missing logging
- **MEDIUM:** Missing versioning, no Multi-AZ (availability, not security)

This is the standard approach for production systems.

Exercise 2: When to Scan Infrastructure

Scenario: You need to decide when to scan Terraform code.

Question: When should scanning happen?

Options:

- A) After terraform apply (production)
- B) Before terraform apply (staging)
- C) During terraform plan (before apply)
- D) All of the above

Think about:

- What's the cost of finding misconfigurations late?
- What's the benefit of early detection?
- What's the industry standard?

Your answer: _____

ANSWER BELOW 

Answer: C) During terraform plan (before apply)

Scanning Timeline:

- **AFTER DEPLOYMENT (Production):** Misconfigurations already in production, infrastructure exposed to risk, requires emergency fix → Too late!
- **BEFORE DEPLOYMENT (Staging):** Misconfigurations found late, requires code changes, slows deployment → Better than production, but still late
- **DURING PLAN (Before apply):** Misconfigurations found immediately, fix before infrastructure is created, fast feedback loop → Industry standard
- **ALL OF THE ABOVE (Defense in depth):** Plan checks block bad code, pre-commit hooks catch issues early, production monitoring (optional) → Best practice

Recommended Approach:

1. **Pre-commit hook** - Fast feedback, catch issues early
2. **During terraform plan** - Block insecure infrastructure automatically
3. **In CI/CD** - Automated validation before merge
4. **In production** (optional) - Continuous monitoring

This is the "shift left" principle: catch issues early.

Design Checkpoint

Before moving on, make sure you can answer these in plain English:

Question	Your Answer Should Be Like...
What is infrastructure scanning?	"Scans Terraform code for security misconfigurations"
Why use tfsec?	"Catches issues in infrastructure that Trivy and SonarCloud don't find"
When to scan?	"During terraform plan - catch issues before apply"
What's a critical issue?	"Public S3 buckets, unencrypted databases, permissive security groups"

Can't answer one? Go back and re-read that exercise.

All clear? Great! Let's install tfsec and start scanning!

Phase 2: Implementation (1.5 hours)

What We're Building

Before this task:

- No infrastructure scanning
- Unknown misconfigurations in Terraform
- Hope-based security

After this task:

- tfsec scanning locally
- Pre-commit hooks preventing insecure code
- Misconfigurations caught before apply
- Security integrated into workflow

Step-by-Step Plan

We'll do this in 5 steps:

Step 1: Install tfsec (10 min)

"Get the scanning tool"

Step 2: Scan Terraform Code (15 min)

"Test scanning on existing infrastructure"

Step 3: Fix Common Issues (30 min)

"Update Terraform to resolve misconfigurations"

Step 4: Create Pre-commit Hook (15 min)

"Automate scanning in workflow"

Step 5: Verify & Test (20 min)

"Confirm scanning works end-to-end"

Step 1: Install tfsec

What we're doing: Installing tfsec on your local machine.

Installation (Mac):

```
# Install via Homebrew
brew install tfsec

# Verify installation
tfsec --version

# Expected output:
# v1.28.1
```

Installation (Linux):

```
# Download and install
wget
https://github.com/aquasecurity/tfsec/releases/download/v1.28.1/tfsec
-linux-amd64 -O tfsec
chmod +x tfsec
sudo mv tfsec /usr/local/bin/

# Verify installation
tfsec --version
```

Installation (Windows - WSL2 or Docker):

```
# Option 1: Use Docker (recommended)
docker run --rm -it -v "$(pwd):/src" aquasec/tfsec:latest /src

# Option 2: Use WSL2 (follow Linux instructions)
# (Same as Linux installation above)
```

Verify tfsec works:

```
# Navigate to Terraform directory
cd ~/projects/h-game-terraform

# Test scan
tfsec --version

# Expected output:
# v1.28.1
```

Common Installation Issues:**Issue 1: "command not found: tfsec"**

```
# Mac: Restart terminal
# Or run: source ~/.zshrc

# Linux: Restart terminal
# Or run: source ~/.bashrc

# Verify: which tfsec
# Should show: /usr/local/bin/tfsec (or similar)
```

Issue 2: "permission denied" (Linux)

```
# Option 1: Use sudo
sudo mv tfsec /usr/local/bin/

# Option 2: Use Docker (no sudo needed)
docker run --rm -it -v "$(pwd):/src" aquasec/tfsec:latest /src
```

Issue 3: Homebrew not found (Mac)

```
# Install Homebrew first
/bin/bash -c "$(curl -fsSL
https://raw.githubusercontent.com/Homebrew/install/HEAD/install.sh)"

# Then install tfsec
brew install tfsec
```

Checkpoint - tfsec Installed:

- tfsec installed successfully
- Version command works
- Ready to scan

tfsec installed!

Step 2: Scan Terraform Code

What we're doing: Scanning your existing H-Game Terraform code for misconfigurations.

Basic scan:

```
# Navigate to Terraform directory
cd ~/projects/h-game-terraform

# Scan all Terraform files
tfsec .
```

Expected output:

```
# 2024/01/15 10:30:45 [INFO] Loading terraform code...
# 2024/01/15 10:30:46 [INFO] Running checks...
#
# Result #1 HIGH
#
-----
# [aws-s3-block-public-acls][HIGH] Resource 'aws_s3_bucket.state' has
public access enabled
# /path/to/terraform/s3.tf:5
#
# Result #2 HIGH
#
-----
# [aws-rds-enable-performance-insights][MEDIUM] Resource
'aws_db_instance.postgres' does not have performance insights enabled
# /path/to/terraform/rds.tf:12
```

Scan with severity filter:

```
# Only show CRITICAL and HIGH
tfsec --minimum-severity HIGH .
```


Expected output:

```
# Result #1 HIGH
#
_____

# [aws-s3-block-public-acls][HIGH] Resource 'aws_s3_bucket.state' has
public access enabled
```

Scan and save results:

```
# Save results as JSON
tfsec --format json --out tfsec-results.json .

# Save results as JUnit XML (for CI/CD)
tfsec --format junit --out tfsec-results.xml .

# View JSON results
cat tfsec-results.json | jq '.results[] | {severity, rule_id,
message}'
```

Scan specific file:

```
# Scan only RDS configuration
tfsec rds.tf

# Scan only security groups
tfsec security-groups.tf
```

Understanding scan results:

Each issue shows:

- **Severity:** CRITICAL, HIGH, MEDIUM, or LOW
- **Rule ID:** Unique identifier (e.g., aws-s3-block-public-acls)
- **Message:** What the issue is (e.g., "S3 bucket has public access enabled")
- **File:** Which Terraform file (e.g., s3.tf:5)

- **Fix:** How to resolve (usually shown in documentation)

Checkpoint - Scanning Works:

- ☐ Terraform code scanned
- ☐ Results saved
- ☐ Misconfigurations identified

Terraform scanned!

Step 3: Fix Common Issues

What we're doing: Updating Terraform code to fix security misconfigurations.

Common fixes:

Fix 1: S3 bucket public access

```
# BEFORE (insecure)
resource "aws_s3_bucket" "logs" {
  bucket = "h-game-logs"
}

# AFTER (secure)
resource "aws_s3_bucket" "logs" {
  bucket = "h-game-logs"
}

resource "aws_s3_bucket_public_access_block" "logs" {
  bucket = aws_s3_bucket.logs.id

  block_public_acls       = true
  block_public_policy     = true
  ignore_public_acls     = true
  restrict_public_buckets = true
}
```


Fix 2: RDS encryption

```
# BEFORE (insecure)
resource "aws_db_instance" "postgres" {
  identifier      = "h-game-db"
  engine         = "postgres"
  engine_version = "15.4"
  instance_class = "db.t3.micro"
  # ... other config ...
}

# AFTER (secure)
resource "aws_db_instance" "postgres" {
  identifier      = "h-game-db"
  engine         = "postgres"
  engine_version = "15.4"
  instance_class = "db.t3.micro"
  storage_encrypted = true # ← Add this
  # ... other config ...
}
```

Fix 3: Security group too permissive

```
# BEFORE (insecure)
resource "aws_security_group_rule" "ssh" {
  type      = "ingress"
  from_port = 22
  to_port   = 22
  protocol  = "tcp"
  cidr_blocks = ["0.0.0.0/0"] # Open to world
  # ... other config ...
}

# AFTER (secure)
resource "aws_security_group_rule" "ssh" {
  type      = "ingress"
  from_port = 22
}
```

```

    to_port          = 22
    protocol         = "tcp"
    security_groups  = [aws_security_group.bastion.id] # Only from
bastion
    # ... other config ...
}

```

Fix 4: Missing CloudTrail

```

# BEFORE (insecure)
# No CloudTrail configured

# AFTER (secure)
resource "aws_cloudtrail" "main" {
    name          = "h-game-cloudtrail"
    s3_bucket_name = aws_s3_bucket.cloudtrail.id

    enable_logging          = true
    include_global_service_events = true
    is_multi_region_trail  = true
}

```

Update your Terraform files:

```

# Navigate to Terraform directory
cd ~/projects/h-game-terraform

# Run tfsec to see what needs fixing
tfsec --minimum-severity HIGH .

# Fix issues one by one
# Edit the Terraform files shown in scan results

```

Re-scan after fixes:

```
# Re-scan to verify fixes
tfsec --minimum-severity HIGH .

# Expected: Fewer or no HIGH/CRITICAL issues
```

Checkpoint - Issues Fixed:

- ☐ Terraform files updated
- ☐ Re-scanned
- ☐ HIGH/CRITICAL issues resolved

Misconfigurations fixed!

Step 4: Create Pre-commit Hook

What we're doing: Creating a pre-commit hook to automatically scan Terraform before commits.

Create pre-commit hook:

```
# Navigate to Terraform directory
cd ~/projects/h-game-terraform

# Create .git/hooks directory if it doesn't exist
mkdir -p .git/hooks
```

```
# Create pre-commit hook
cat > .git/hooks/pre-commit << 'EOF'
#!/bin/bash
# =====
# TERRAFORM SECURITY SCANNING PRE-COMMIT HOOK
# =====
# Scans Terraform files before commit
# Prevents insecure code from being committed

set -e

# Check if tfsec is installed
if ! command -v tfsec &> /dev/null; then
    echo "  tfsec not found. Install with: brew install tfsec"
    exit 0 # Don't block commit if tfsec not installed
fi

# Get list of staged Terraform files
STAGED_FILES=$(git diff --cached --name-only --diff-filter=ACM | grep
'\.tf$' || true)

if [ -z "$STAGED_FILES" ]; then
    # No Terraform files staged, skip scan
    exit 0
fi
```

```

echo "🔍 Scanning Terraform files with tfsec..."

# Run tfsec on staged files
# --soft-fail: Don't exit with error (just warn)
# --minimum-severity: Only show HIGH and CRITICAL
tfsec --soft-fail --minimum-severity HIGH $STAGED_FILES

SCAN_EXIT_CODE=$?

if [ $SCAN_EXIT_CODE -eq 0 ]; then
    echo " Terraform scan passed: No HIGH/CRITICAL issues found"
    exit 0
else
    echo " Terraform scan found HIGH/CRITICAL issues"
    echo " Review the issues above and fix before committing"
    echo " To commit anyway: git commit --no-verify"
    exit 1 # Block commit
fi
EOF

```

Make hook executable

```

chmod +x .git/hooks/pre-commit

```

Test pre-commit hook:

```

# Make a test change to Terraform
echo "# Test comment" >> vpc.tf

# Stage the change
git add vpc.tf

# Try to commit (should trigger scan)
git commit -m "test: trigger tfsec scan"

# Expected: tfsec runs, shows results
# If issues found: Commit blocked
# If no issues: Commit succeeds

```

Create scan script (optional):

```
# Create scripts directory
mkdir -p scripts
```

Create scan script

```
cat > scripts/scan-terraform.sh << 'EOF'
#!/bin/bash
# =====
# TERRAFORM SECURITY SCANNING SCRIPT
# =====
# Scans Terraform code with tfsec
# Exits with error code if critical/high issues found

set -e

TERRAFORM_DIR=${1:-.}
SEVERITY=${2:-HIGH} # Default: HIGH

if [ ! -d "$TERRAFORM_DIR" ]; then
    echo "Usage: $0 <terraform-directory> [severity]"
    echo "Example: $0 terraform/ HIGH"
    exit 1
fi

echo " Scanning Terraform: $TERRAFORM_DIR"
echo " Minimum severity: $SEVERITY"

# Run tfsec scan
tfsec --minimum-severity "$SEVERITY" "$TERRAFORM_DIR"

SCAN_EXIT_CODE=$?

if [ $SCAN_EXIT_CODE -eq 0 ]; then
    echo " Terraform scan passed: No $SEVERITY issues found"
    exit 0
```

```
else
  echo " Terraform scan failed: $SEVERITY issues found"
  echo " Fix issues and re-scan"
  exit 1
fi
EOF
```

Make script executable

```
chmod +x scripts/scan-terraform.sh
```

Checkpoint - Automation Created:

- Pre-commit hook created
- Scan script created
- Tested successfully
- Automation ready

Automation configured!

Step 5: Verify & Test

What we're doing: Testing the complete scanning workflow.

Test 1: Local scan

```
# Navigate to Terraform directory
cd ~/projects/h-game-terraform

# Scan with script
./scripts/scan-terraform.sh .

# Expected: Scan results, exit code based on issues
```

Test 2: Pre-commit hook

```
# Make a test change
echo "# Test" >> test.tf

# Stage change
git add test.tf

# Try to commit
git commit -m "test: verify pre-commit hook"

# Expected: tfsec runs automatically
# If issues: Commit blocked
# If no issues: Commit succeeds
```

Test 3: Fail on critical issues

```
# Create a test file with insecure config
cat > test-insecure.tf << 'EOF'
resource "aws_s3_bucket" "test" {
  bucket = "test-public-bucket"
  # Missing public_access_block - tfsec will flag this
}
```

```
EOF
```

Scan it

```
tfsec test-insecure.tf

# Expected: Exit code 1, HIGH issue found

# Clean up
rm test-insecure.tf
```

Test 4: Verify fixes work

```
# After fixing issues, re-scan
tfsec --minimum-severity HIGH .

# Expected: No HIGH/CRITICAL issues (or reduced count)
```

Checkpoint - Complete Workflow Verified:

- Local scanning works
- Pre-commit hook works
- Scan results accurate
- Build fails on critical issues

Scanning workflow complete!

Step 6: Commit Your Work

What we're doing: Committing security improvements to Git.

```
# Navigate to Terraform directory
cd ~/projects/h-game-terraform

# Add scan script
git add scripts/scan-terraform.sh

# Add pre-commit hook (if you want to commit it)
# Note: .git/hooks/ is usually not committed
# Consider using a hooks manager like pre-commit framework

# Add fixed Terraform files
git add *.tf

# Commit with clear message
git commit -m "feat(security): Add tfsec infrastructure scanning

- Install tfsec for Terraform security scanning
- Fix S3 public access issues
- Fix RDS encryption configuration
- Add pre-commit hook for automated scanning
- Create scan script for CI/CD

Closes: DEVOPS-6102"

# Push to remote
git push origin main
```

Checkpoint - Work Committed:

- Scan script committed
- Terraform fixes committed
- Clear commit message
- Changes pushed to remote

Work committed!

Phase 3: Validation & Consolidation (20 min)

Quiz: Infrastructure Security Understanding

Answer these questions to verify your understanding:

Question 1: What does tfsec scan that Trivy and SonarCloud don't?

Answer:

- **tfsec** scans: Terraform code for infrastructure misconfigurations
- **Trivy** scans: Docker images for vulnerabilities in base images and dependencies
- **SonarCloud** scans: Application code (JavaScript, Node.js) for bugs and vulnerabilities

Key point: Each tool scans a different layer. You need all three for complete security.

Question 2: What's a critical infrastructure misconfiguration?

Answer:

- **Public S3 buckets** - Data exposed to internet
- **Unencrypted databases** - RDS without encryption
- **Permissive security groups** - Open to 0.0.0.0/0

Key point: Critical issues expose data or allow unauthorized access.

Question 3: When should infrastructure scanning happen?

Answer:

- **During terraform plan** - Primary scanning point (before apply)
- **Pre-commit** - Local validation
- **In CI/CD** - Automated validation before merge

Key point: Multiple layers of scanning = Defense in depth.

Knowledge Check

Test your understanding before moving on:

1. What does tfsec scan?

- ☐ a) Docker images
- ☐ b) Terraform code
- ☐ c) Application code

2. What's a critical infrastructure issue?

- ☐ a) Missing versioning
- ☐ b) Public S3 bucket
- ☐ c) No Multi-AZ

3. When should you scan Terraform?

- ☐ a) Only in production
- ☐ b) During terraform plan
- ☐ c) Once a month

4. How do you fix a public S3 bucket?

- ☐ a) Delete the bucket
- ☐ b) Add `aws_s3_bucket_public_access_block`
- ☐ c) Change the bucket name

5. Is tfsec free?

- ☐ a) No, it costs money
- ☐ b) Yes, open source
- ☐ c) Only for students

Score:

- 5/5: Excellent! Move to D2.5
- 3-4/5: Review weak areas
- <3/5: Re-read Phase 1

Notes

1. tfsec scans Terraform code for infrastructure misconfigurations - different from Trivy (containers) and SonarCloud (application code)
2. Public S3 bucket = critical security issue (data exposed to internet)
3. Scan during terraform plan to catch issues before apply (shift left)
4. Add `aws_s3_bucket_public_access_block` resource to block public access
5. tfsec is free and open source (no cost)

Checklist: Did You Complete Everything?

Before moving on, check these boxes:

Understanding Phase

- ☐ I understand what infrastructure misconfigurations are
- ☐ I know why tfsec matters
- ☐ I understand when to scan
- ☐ I can explain common security issues

Implementation Phase

- ☐ tfsec installed
- ☐ Terraform code scanned
- ☐ Misconfigurations identified
- ☐ Terraform files updated (if needed)
- ☐ Pre-commit hook created
- ☐ Scan script created
- ☐ Workflow tested end-to-end

Verification Phase

- ☐ Can explain what tfsec scans
- ☐ Can scan Terraform locally
- ☐ Can fix common misconfigurations
- ☐ Can integrate scanning into workflow

Git

- ☐ Committed scan script
- ☐ Updated Terraform files (if fixes applied)
- ☐ Documented scanning process

Key Takeaways

The Big Picture

What you learned in this task:

- tfsec installed and configured
- Local scanning working
- Pre-commit hooks preventing insecure code
- Misconfigurations caught before terraform apply
- Security integrated into workflow

Infrastructure code is now scanned automatically!

Critical Concepts

1. Infrastructure Misconfigurations:

- Exist in Terraform code (S3, RDS, security groups)
- Can expose data or allow unauthorized access
- Severity: CRITICAL > HIGH > MEDIUM > LOW

2. Scanning Strategy:

- Scan during terraform plan - primary
- Pre-commit hooks - catch issues early
- Block on CRITICAL/HIGH
- Warn on MEDIUM/LOW

3. Common Issues:

- Public S3 buckets (critical)
- Unencrypted databases (critical)
- Permissive security groups (high)
- Missing encryption (high)

4. Automation:

- Pre-commit hooks automate checking
- Scan scripts integrate into CI/CD
- tfsec scans automatically before apply

What's Next?

Congratulations! You've secured your infrastructure code!

You now have:

- Infrastructure scanning automated
- Misconfigurations caught before deployment
- Pre-commit hooks preventing insecure code
- Security integrated into workflow

Next: TASK D2.5 - Code Quality & Security (SonarCloud)

In D2.5, you'll learn:

- How to scan application code for bugs and vulnerabilities
- How to integrate SonarCloud with GitHub Actions
- How all security layers work together (Trivy + tfsec + SonarCloud)

But most importantly: You now understand infrastructure security. Combined with container and code scanning, you have complete security coverage!

Proceed to TASK D2.5: Code Quality & Security (SonarCloud)

D2.5: Code Quality & Security Scanning (SonarCloud)

Time: 1-2 hours | **Priority:** High | **Ticket:** DEVOPS-6102.5 | **Cost:** Free

Mission Brief

Scan your application code for bugs, vulnerabilities, and code smells using SonarCloud. Catch security issues in your JavaScript/Node.js code before they reach production.

Outcomes:

- Understand code quality vs security scanning
- Set up SonarCloud for your repository
- Configure automated code analysis
- Fix common code quality issues
- Integrate SonarCloud into your workflow

Prerequisites: Complete D1 (Container Scanning) and D2 (Infrastructure Scanning)

Setup Note: SonarCloud runs in the cloud. You connect your GitHub repository and it scans automatically.

Cost Note: This task costs \$0. SonarCloud is free for open source/public repositories.

The One Thing That Makes This Click

Code quality scanning is like **spell check and grammar check for your application code**.

Just as spell check catches mistakes before you send an email, **SonarCloud** catches bugs and security problems **before your code is deployed**.

Without code scanning, developers usually write code, test a little, and then deploy while **hoping nothing is wrong**.

Bugs and security issues often appear later in production, where they are expensive, stressful, and risky to fix.

With **SonarCloud**, every code change is scanned automatically when you push code or open a pull request.

If SonarCloud finds bugs, security vulnerabilities, or bad coding practices, it **blocks the code from being merged** until the issues are fixed.

How SonarCloud Works with Other Security Tools

Security is not handled by one tool. Each tool scans a **different layer** of your system:

- **Trivy** scans Docker images and dependencies for known vulnerabilities
- **tfsec** scans Terraform code for unsafe infrastructure settings
- **SonarCloud** scans your application code (Node.js, JavaScript, etc.)

Each tool looks at something different.

Example:

If your Node.js code contains a hardcoded password like:

```
const password = "admin123";
```

- **Trivy** will not catch it (it scans images and dependencies)
- **tfsec** will not catch it (it scans infrastructure)
- **SonarCloud** will catch it and flag it as a security issue

This is why **SonarCloud is necessary**, even if you already use Trivy and tfsec.

Mental Model Shift

Before Code Scanning

- Code is “secure” if it runs
- Bugs are found after deployment
- Security issues appear in production
- Fixes are rushed and stressful

After SonarCloud

- Every commit is scanned automatically
- Bugs are caught before merge
- Security issues are fixed early
- Production is safer and more stable

Why This Matters in Real Life

With **SonarCloud**, you can show:

- Automated code quality checks
- Security scanning reports
- Clear evidence of secure coding practices

This is important for:

- Production stability
- Security audits
- Job interviews

Instead of saying “*I try to write good code*”, you can say:

“Every line of code is automatically scanned before deployment.”

One-Sentence Takeaway

SonarCloud turns code security from a guessing game into an automated safety check that runs on every commit.

Exercise 1: Understanding Code Quality vs Security

Scenario: SonarCloud finds issues in your code.

Question: What's the difference between a "Bug" and a "Security Hotspot"?

Think about:

- What causes bugs vs security issues?
- What's the impact of each?
- How do you prioritize fixes?

Your answer: _____

Answer Below 

Answer:**Bug:**

- Code that doesn't work as intended
- Example: Null pointer exception, infinite loop
- Impact: Application crashes, functionality broken
- Priority: Fix before production (breaks functionality)

Security Hotspot:

- Code that could be exploited
- Example: Hardcoded password, SQL injection risk
- Impact: Data breach, unauthorized access
- Priority: Fix immediately (security risk)

Vulnerability:

- Confirmed security issue
- Example: Known CVE in code pattern
- Impact: Exploitable security flaw
- Priority: Fix immediately (critical)

Code Smell:

- Code that works but is hard to maintain
- Example: Duplicated code, complex functions
- Impact: Technical debt, harder to maintain
- Priority: Fix when convenient (quality improvement)

Production Policy:

- Fix: Bugs, Security Hotspots, Vulnerabilities (before merge)
- Address: Code Smells (in next sprint)

This is the standard approach for production systems.

Exercise 2: When to Scan Code

Scenario: You need to decide when to scan application code.

Question: When should code scanning happen?

Options:

- A) After deployment (production)
- B) Before merge (pull request)
- C) On every commit
- D) All of the above

Think about:

- What's the cost of finding bugs late?
- What's the benefit of early detection?
- What's the industry standard?

Your answer: _____

Answer Below 

Answer: D) All of the above

Scanning Timeline:

- **(wrong) AFTER DEPLOYMENT (Production):** Bugs already in production, users affected, requires emergency fix → Too late!
- **(good) BEFORE MERGE (Pull Request):** Bugs found before code is merged, fast feedback, prevents bad code → Industry standard
- **(good) ON EVERY COMMIT:** Continuous feedback, catch issues immediately, fast iteration → Best practice
- **(good) ALL OF THE ABOVE (Defense in depth):** PR checks block bad code, commit scans provide feedback, production monitoring catches edge cases → Comprehensive coverage

Recommended Approach:

1. **On commit:** Fast feedback, catch issues immediately
2. **On pull request:** Block bad code before merge
3. **On merge:** Final validation before deployment
4. **In production (optional):** Continuous monitoring

This is the "shift left" principle: catch issues early.

Design Checkpoint

Before moving on, make sure you can answer these in plain English:

Question	Your Answer Should Be Like...
What is code quality scanning?	"Scans application code for bugs, vulnerabilities, and code smells"
Why use SonarCloud?	"Catches issues in code that Trivy and tfsec don't find"
When to scan?	"On every commit and before merge - catch issues early"
What's a security hotspot?	"Code that could be exploited, needs immediate fix"

Can't answer one? Go back and re-read that exercise.

All clear? Great! Let's set up SonarCloud!

Phase 2: Implementation (1 hour)

What We're Building

Before this task:

- No code quality scanning
- Unknown bugs in application code
- No visibility into code security

After this task:

- SonarCloud scanning on every commit
- Automated PR checks
- Code quality metrics visible
- Security hotspots identified

Step-by-Step Plan

We'll do this in 5 steps:

Step 1:

Create SonarCloud Account (10 min)

"Sign up and connect GitHub"

Step 2:

Add Project to SonarCloud (10 min)

"Connect your repository"

Step 3:

Configure SonarCloud Analysis (15 min)

"Set up automated scanning"

Step 4:

Fix First Issues (20 min)

"Address code quality problems"

Step 5:

Verify & Integrate (10 min)

"Confirm scanning works end-to-end"

Step 1: Create SonarCloud Account

What we're doing: Creating a free SonarCloud account and connecting it to GitHub.

Sign up for SonarCloud:

1. Go to <https://sonarcloud.io>
2. Click "Log in" → "Log in with GitHub"
3. Authorize SonarCloud to access your GitHub account
4. Complete the setup wizard

Expected: You'll be redirected to SonarCloud dashboard after GitHub authorization.

Verify account:

- # Check if you can access SonarCloud
- # You should see the SonarCloud dashboard

Checkpoint - Account Created:

- ☐ SonarCloud account created
- ☐ GitHub connected
- ☐ Dashboard accessible

SonarCloud account ready!

Step 2: Add Project to SonarCloud

What we're doing: Adding your H-Game repository to SonarCloud for analysis.

Add project:

1. In SonarCloud dashboard, click "Create Project"
2. Select "GitHub" as your organization
3. Choose your H-Game repository
4. Click "Set Up" on the project

Configure project:

- **Project Key:** Auto-generated (e.g., `yourusername_h-game`)
- **Project Name:** H-Game
- **Main Branch:** main (or master)

Expected: Project appears in SonarCloud dashboard.

Checkpoint - Project Added:

- ☐ Repository connected
- ☐ Project created in SonarCloud
- ☐ Ready for analysis

Project added!

Step 3: Configure SonarCloud Analysis

What we're doing: Setting up automated code analysis using GitHub Actions.

Create SonarCloud token:

1. In SonarCloud, go to: My Account → Security
2. Generate a new token
3. Name it: "GitHub Actions"
4. Copy the token (you'll need it)

Add token to GitHub Secrets:

1. Go to your GitHub repository
2. Settings → Secrets and variables → Actions
3. Click "New repository secret"
4. Name: `SONAR_TOKEN`
5. Value: Paste your SonarCloud token
6. Click "Add secret"

Create GitHub Actions workflow:

```
# Navigate to project root
cd ~/projects/H-Game

# Create .github/workflows directory
mkdir -p .github/workflows

# Create SonarCloud workflow
cat > .github/workflows/sonarcloud.yml << 'EOF'
name: SonarCloud Analysis

on:
  push:
    branches:
      - main
      - develop
  pull_request:
    types: [opened, synchronize, reopened]

jobs:
```

```

sonarcloud:
  name: SonarCloud Scan
  runs-on: ubuntu-latest

  steps:
    - name: Checkout code
      uses: actions/checkout@v3
      with:
        fetch-depth: 0 # Shallow clones should be disabled for
        better analysis

    - name: Setup Node.js
      uses: actions/setup-node@v3
      with:
        node-version: '18'
        cache: 'npm'

    - name: Install dependencies
      run: npm ci
      working-directory: ./backend

    - name: Run SonarCloud Scan
      uses: SonarSource/sonarcloud-github-action@master
      env:
        GITHUB_TOKEN: ${ secrets.GITHUB_TOKEN }
        SONAR_TOKEN: ${ secrets.SONAR_TOKEN }
EOF

```

Commit and push:

```

# Add workflow file
git add .github/workflows/sonarcloud.yml

# Commit
git commit -m "feat(ci): Add SonarCloud code quality scanning

- Configure SonarCloud analysis

```

- Scan on push and pull requests
- Catch bugs and security issues early

Closes: DEVOPS-6102.5"

```
# Push to trigger first scan  
git push origin main
```

Expected: GitHub Actions runs and SonarCloud analysis starts.

Checkpoint - Analysis Configured:

- ☐ SonarCloud token created
- ☐ GitHub secret added
- ☐ Workflow file created
- ☐ First scan triggered

Analysis configured!

Step 4: Fix First Issues

What we're doing: Addressing code quality issues found by SonarCloud.

View results:

1. Go to SonarCloud dashboard
2. Click on your H-Game project
3. Review the issues found

Common issues you might see:

Issue 1: Hardcoded values

```
// BEFORE (SonarCloud flags this)
const API_KEY = "sk-1234567890";

// AFTER (secure)
const API_KEY = process.env.API_KEY;
```

Issue 2: SQL injection risk

```
// BEFORE (SonarCloud flags this)
const query = `SELECT * FROM users WHERE id = ${userId}`;

// AFTER (secure)
const query = 'SELECT * FROM users WHERE id = $1';
await db.query(query, [userId]);
```

Issue 3: Unused variables

```
// BEFORE (SonarCloud flags this)
const unusedVar = "test";

// AFTER (remove or use)
```

Issue 4: Code duplication

```
// BEFORE (SonarCloud flags this)
```



```
function calculateTotal(items) {
  let total = 0;
  for (let i = 0; i < items.length; i++) {
    total += items[i].price;
  }
  return total;
}

// AFTER (refactored)
function calculateTotal(items) {
  return items.reduce((sum, item) => sum + item.price, 0);
}
```

Fix issues:

1. Review SonarCloud dashboard for your project
2. Click on "Issues" tab
3. Filter by severity: Start with "Blocker" and "Critical"
4. Fix issues in your code
5. Commit and push changes
6. SonarCloud re-scans automatically

Expected: Issue count decreases after fixes.

Checkpoint - Issues Fixed:

- ☐ SonarCloud results reviewed
- ☐ Critical issues identified
- ☐ Code fixes applied
- ☐ Re-scan shows improvements

Issues fixed!

Step 5: Verify & Integrate

What we're doing: Verifying SonarCloud works end-to-end and integrating into workflow.

Verify PR checks:

1. Create a test branch
2. Make a code change
3. Create a pull request
4. Check that SonarCloud analysis runs
5. Verify results appear in PR

Expected: SonarCloud analysis runs on PR and shows results.

View quality gate:

1. In SonarCloud dashboard
2. Check "Quality Gate" status
3. Should show: Passed (green) or Failed (red)

Quality Gate criteria:

- Coverage: Code coverage percentage
- Duplications: Code duplication percentage
- Maintainability: Technical debt ratio
- Reliability: Bugs count
- Security: Vulnerabilities and security hotspots

Set up quality gate enforcement (optional):

```
# Add to .github/workflows/sonarcloud.yml
# This will fail the build if quality gate fails
- name: Check Quality Gate
  uses: sonarsource/sonarcloud-github-action@master
  env:
    GITHUB_TOKEN: ${ secrets.GITHUB_TOKEN }
    SONAR_TOKEN: ${ secrets.SONAR_TOKEN }
```

Checkpoint - Complete Workflow Verified:

- ☐ SonarCloud scans on commits
- ☐ PR checks working
- ☐ Quality gate visible
- ☐ Issues being tracked

SonarCloud integrated!

Step 6: Commit Your Work

What we're doing: Committing SonarCloud configuration to Git.

```
# Navigate to project root
cd ~/projects/H-Game

# Add workflow file
git add .github/workflows/sonarcloud.yml

# Commit with clear message
git commit -m "feat(security): Add SonarCloud code quality scanning

- Configure SonarCloud for automated code analysis
- Scan on push and pull requests
- Catch bugs and security hotspots early
- Integrate with GitHub Actions

Closes: DEVOPS-6102.5"

# Push to remote
git push origin main
```

Checkpoint - Work Committed:

- ☐ Workflow file committed
- ☐ Clear commit message
- ☐ Changes pushed to remote

Work committed!

Phase 3: Validation & Consolidation (20 min)

Quiz: Code Quality Understanding

Answer these questions to verify your understanding:

Question 1: What does SonarCloud scan that Trivy and tfsec don't?

Answer:

- **SonarCloud** scans: Application code (JavaScript, Node.js) for bugs, vulnerabilities, and code smells
- **Trivy** scans: Docker images for vulnerabilities in base images and dependencies
- **tfsec** scans: Terraform code for infrastructure misconfigurations

Key point: Each tool scans a different layer. You need all three for complete security.

Question 2: What's the difference between a bug and a security hotspot?

Answer:

- **Bug:** Code that doesn't work as intended (crashes, broken functionality)
- **Security Hotspot:** Code that could be exploited (hardcoded secrets, SQL injection risk)

Key point: Both need fixing, but security hotspots are higher priority.

Question 3: When should code scanning happen?

Answer:

- **On every commit** - Fast feedback, catch issues immediately
- **On pull request** - Block bad code before merge
- **On merge** - Final validation before deployment

Key point: Multiple layers of scanning = Defense in depth.

Knowledge Check

Test your understanding before moving on:

1. What does SonarCloud scan?

- ☐ a) Docker images
- ☐ b) Terraform code
- ☐ c) Application code

2. What's a security hotspot?

- ☐ a) A bug that crashes the app
- ☐ b) Code that could be exploited
- ☐ c) Code duplication

3. When should SonarCloud scan?

- ☐ a) Only in production
- ☐ b) On every commit and PR
- ☐ c) Once a month

4. What's the quality gate?

- ☐ a) A security checkpoint
- ☐ b) Code quality standards that must pass
- ☐ c) A GitHub feature

5. Is SonarCloud free?

- ☐ a) No, it costs money
- ☐ b) Yes, for open source projects
- ☐ c) Only for students

Score:

- 5/5: Excellent! Move to D3
- 3-4/5: Review weak areas
- <3/5: Re-read Phase 1

Explanations

1. **SonarCloud scans application code (JavaScript, Node.js)** - different from Trivy (containers) and tfsec (infrastructure)
2. **Security hotspot** = code that could be exploited (needs immediate fix)
3. **Scan on every commit** and PR to catch issues early (shift left)
4. **Quality gate** = code quality standards (coverage, bugs, security) that must pass
5. **SonarCloud** is free for open source/public repositories

Checklist: Did You Complete Everything?

Before moving on, check these boxes:

Understanding Phase

- ☐ I understand what code quality scanning is
- ☐ I know why SonarCloud matters
- ☐ I understand when to scan
- ☐ I can explain bugs vs security hotspots

Implementation Phase

- ☐ SonarCloud account created
- ☐ Repository connected
- ☐ GitHub Actions workflow created
- ☐ First scan completed
- ☐ Issues reviewed and fixed
- ☐ PR checks working

Verification Phase

- ☐ Can explain what SonarCloud scans
- ☐ Can view SonarCloud dashboard
- ☐ Can fix code quality issues
- ☐ Can integrate into workflow

Git

- ☐ Committed workflow file
- ☐ Documented SonarCloud setup

Key Takeaways

The Big Picture

What you learned in this task:

- ☐ SonarCloud account created and configured
- ☐ Repository connected for automated scanning
- ☐ GitHub Actions workflow set up
- ☐ Code quality issues identified and fixed
- ☐ Security hotspots caught early

Application code is now scanned automatically!

Critical Concepts

1. Code Quality Scanning:

- Scans application code (JavaScript, Node.js)
- Finds bugs, vulnerabilities, code smells
- Different from container and infrastructure scanning

2. Three Layers of Security:

- Trivy: Container images
- tfsec: Infrastructure code
- SonarCloud: Application code

3. Quality Gate:

- Code quality standards that must pass
- Coverage, bugs, security metrics
- Can block merges if failed

4. Automation:

- Scans on every commit
- PR checks prevent bad code
- Continuous feedback loop

What's Next?

Congratulations! You've secured your application code!

You now have:

- Code quality scanning automated
- Bugs caught before merge
- Security hotspots identified
- SonarCloud integrated into workflow

Next: TASK D3 - Secrets Management

In D3, you'll learn:

- How to store secrets securely in AWS Secrets Manager
- How to auto-rotate credentials
- How to sync secrets to Kubernetes
- How all security layers work together

But most importantly: You now understand code quality scanning. Combined with container and infrastructure scanning, you have complete security coverage!

Proceed to TASK D3: Secrets Management

TASK D3: Secrets Management

Time: 3 hours | **Priority:** High | **Ticket:** DEVOPS-6103 | **Cost:** ~\$0.50/month

Mission Brief

Remove all hardcoded secrets from code. Store secrets in AWS Secrets Manager, auto rotate them, and inject into Pods securely using External Secrets Operator.

Outcomes:

- Understand secrets management best practices
- Store secrets in AWS Secrets Manager
- Configure automatic secret rotation
- Sync secrets to Kubernetes using External Secrets Operator
- Remove hardcoded secrets from code

Prerequisites: Complete D1 (Container Scanning) and D2 (Infrastructure Scanning)

Setup Note: AWS Secrets Manager runs in AWS. External Secrets Operator runs in your Kubernetes cluster.

Cost Note: This task costs ~\$0.50/month. AWS Secrets Manager charges \$0.40 per secret per month + \$0.05 per 10,000 API calls.

The One Thing That Makes This Click

Secrets Management

AWS Secrets Manager is a secure vault for passwords, API keys, and credentials. Instead of writing secrets directly in code or Git, they are stored securely in AWS and fetched only when needed.

Without Secrets Manager

- Secrets are hardcoded or stored in Kubernetes Secrets (base64, not encrypted)
- Secrets may end up in Git history
- Rotation is manual and often forgotten
- No clear audit trail
- High risk if the cluster is compromised

With AWS Secrets Manager

- Secrets are **encrypted at rest (AES-256)**
- Secrets **rotate automatically every 30 days**
- Secrets are **never stored in code or Git**
- Access is controlled using **IAM**
- All access is logged for audits

Why Kubernetes Secrets Alone Are Not Enough

- They are **only base64-encoded**, not encrypted
- Anyone with read access can decode them
- They live inside the cluster (etcd)
- Not ideal for production security requirements

How External Secrets Operator Fits In

First, secrets are stored safely in AWS Secrets Manager.

Kubernetes applications, however, expect secrets to exist inside Kubernetes.

External Secrets Operator acts as the bridge between the two. It automatically copies secrets from AWS Secrets Manager into Kubernetes Secrets.

Your applications keep working the same way, with no code changes.

When a secret is rotated in AWS, Kubernetes is updated automatically.

Result: secrets are stored securely in AWS, but still work naturally inside Kubernetes.

The Core Idea (Mental Model)

Before

- Secrets stored in Kubernetes or code
- Rotation done manually (or forgotten)
- Security is based on hope

After

- Secrets stored in AWS Secrets Manager
- Automatic rotation every 30 days
- Kubernetes always gets the latest secret
- Security is guaranteed by automation

One-Line Takeaway

Secrets Manager + External Secrets Operator means secrets are encrypted, rotated automatically, and never exposed in code or Git.

Now let's see how secrets management works...

Phase 1: Understanding Secrets Management (30 min)

What is Secrets Management?

The Problem with Kubernetes Secrets:

From Section 4 (K8), you learned that Kubernetes Secrets are:

- Base64-encoded (not encrypted)
- Anyone with Secret read access can decode them
- Stored in etcd (cluster database)
- Not suitable for production with strict security requirements

The Solution: AWS Secrets Manager

- Encrypted at rest (AES-256)
- Automatic rotation every 30 days
- Audit logging (who accessed what, when)
- Version history (rollback if needed)
- IAM-controlled access

Why External Secrets Operator?

- Syncs AWS Secrets Manager → Kubernetes Secrets automatically
- Pods still use Kubernetes Secrets (no code changes needed)
- Secrets update automatically when rotated in AWS
- Best of both worlds: Secure storage + Kubernetes integration

Why Secrets Management Matters

Without **AWS Secrets Manager**, secrets are stored in Kubernetes, rotated manually, and often forgotten.

When a secret leaks, you scramble to update everything, causing risk and downtime.

With **AWS Secrets Manager**, secrets are encrypted, rotated automatically, and synced safely into Kubernetes using **External Secrets Operator**.

Audits are easy, updates are automatic, and security issues are handled before they become incidents.

Exercise 1: Understanding Secret Rotation

Scenario: Your database password needs to be rotated every 30 days.

Question: How should rotation work?

Options:

- A) Manual rotation (update password, restart Pods)
- B) Automatic rotation (AWS rotates, Pods get new password)
- C) No rotation (keep same password forever)

Think about:

- What's the security benefit of rotation?
- What's the operational burden of each approach?
- What do production systems do?

Your answer: _____

ANSWER BELOW 

Answer: B) Automatic rotation

Why each option fails/succeeds:

- **A) Manual rotation:** Security benefit (rotated passwords), but high operational burden (remember to rotate, update all services, potential downtime)
- **B) Automatic rotation:** Security benefit + low operational burden (AWS handles it, External Secrets Operator syncs, zero downtime)
- **C) No rotation:** Low operational burden, but security risk (compromised password stays compromised)

Production Policy:

- Rotate secrets every 30-90 days (depending on sensitivity)
- Use automatic rotation when possible
- Audit secret access regularly
- Never reuse old passwords

This is the standard approach for production systems.

Exercise 2: Secret Storage Strategy

Scenario: You need to store database credentials.

Question: Where should secrets be stored?

Options:

- A) Hardcoded in code
- B) Kubernetes Secrets only
- C) AWS Secrets Manager + Kubernetes Secrets (via External Secrets Operator)

Think about:

- What's the security of each approach?
- What's the operational complexity?
- What's the production standard?

Your answer: _____

ANSWER BELOW 

Answer: C) AWS Secrets Manager + Kubernetes Secrets

Why each option fails/succeeds:

- **A) Hardcoded:** Visible in Git, can't rotate, security breach
- **B) Kubernetes Secrets only:** ⚠ Base64-encoded (not encrypted), no auto-rotation, stored in etcd
- **C) AWS Secrets Manager + K8s Secrets:** Encrypted at rest, auto-rotation, audit logs, IAM-controlled, synced to K8s automatically

Best Practice:

- **Store:** AWS Secrets Manager (source of truth)
- **Sync:** External Secrets Operator syncs to Kubernetes Secrets
- **Use:** Pods read from Kubernetes Secrets (no code changes)
- **Rotate:** AWS rotates automatically, External Secrets Operator syncs new value

This is the production standard.

Design Checkpoint

Before moving on, make sure you can answer these in plain English:

Question	Your Answer Should Be Like...
What is secrets management?	"Storing secrets securely with automatic rotation"
Why use AWS Secrets Manager?	"Encrypted storage, auto-rotation, audit logs"
What is External Secrets Operator?	"Syncs AWS Secrets Manager to Kubernetes Secrets automatically"
Why rotate secrets?	"Reduces risk if password is compromised"

Can't answer one? Go back and re-read that exercise.

All clear? Great! Let's set up secrets management!

Phase 2: Implementation (2.5 hours)

What We're Building

Before this task:

- Secrets in Kubernetes (base64-encoded)
- Manual rotation (if at all)
- Secrets might be in Git history

After this task:

- Secrets in AWS Secrets Manager (encrypted)
- Automatic rotation every 30 days
- External Secrets Operator syncing to K8s
- Zero hardcoded secrets

Step-by-Step Plan

We'll do this in **8 steps**:

1. **Create secrets in AWS:** Store passwords in Secrets Manager
2. **Install External Secrets Operator:** Enable Kubernetes syncing
3. **Configure IRSA:** Allow secure access to secrets
4. **Create a SecretStore:** Connect Kubernetes to AWS
5. **Create an ExternalSecret:** Sync AWS secrets into Kubernetes
6. **Update the deployment :** Use the synced secrets in Pods
7. **Enable auto-rotation:** Configure automatic secret rotation
8. **Verify everything works:** Test end-to-end

Step 1: Create Secrets in AWS Secrets Manager ([all code files in repo](#))

What we're doing: Creating secrets in AWS Secrets Manager to replace hardcoded values.

IMPORTANT: By now, your Terraform already creates secrets! Check what exists:

Check existing secrets (from Terraform)

```
aws secretsmanager list-secrets \
  --filters Key=name,Values=h-game \
  --region us-east-1

# Expected: h-game/rds/password and h-game/api/keys (if created)
```

Option A: Use Existing Terraform Secret (RECOMMENDED)

```
# Your Terraform already created: h-game/rds/password
# Just verify it exists:
aws secretsmanager get-secret-value \
  --secret-id h-game/rds/password \
  --region us-east-1 \
  --query SecretString \
  --output text
```

Option B: Create New Secret (if you want a separate one for External Secrets)

```
# Generate a secure password
DB_PASSWORD=$(openssl rand -base64 32)

# Create secret in AWS Secrets Manager
aws secretsmanager create-secret \
  --name h-game/database/password \
  --description "Database credentials for H-Game" \
  --secret-string
"{\"username\": \"gameuser\", \"password\": \"${DB_PASSWORD}\"} \" \
  --region us-east-1

# Expected output:
```

```
# {
#   "ARN":
#   "arn:aws:secretsmanager:us-east-1:123456789012:secret:h-game/database
#   /password-abc123",
#   "Name": "h-game/database/password",
#   "VersionId": "abc123-def456-ghi789"
# }
```

⚠ CRITICAL: If you use **Option B**, you'll have **TWO** secrets:

- **h-game/rds/password** (from Terraform, includes host/port/dbname)
- **h-game/database/password** (new, just username/password)

Recommendation: Use the existing Terraform secret and update ExternalSecret to extract only the fields you need.

Verify secret created:

```
# List secrets
aws secretsmanager list-secrets \
  --filters Key=name,Values=h-game \
  --region us-east-1

# Get secret value (for verification)
aws secretsmanager get-secret-value \
  --secret-id h-game/database/password \
  --region us-east-1 \
  --query SecretString \
  --output text

# Expected output:
# {"username": "gameuser", "password": "<generated-password>"}
```

Create API keys secret (if needed):

```
# Create API keys secret
aws secretsmanager create-secret \
  --name h-game/api/keys \
  --description "API keys for H-Game" \
```

```
--secret-string  
'{"api_key":"your-api-key","api_secret":"your-secret"}' \  
--region us-east-1
```

Checkpoint - Secrets Created:

- ☐ Database secret created in AWS
- ☐ Secret value verified
- ☐ Ready for External Secrets Operator

Secrets created in AWS!

Step 2: Install External Secrets Operator

What we're doing: Installing External Secrets Operator in your Kubernetes cluster to sync AWS secrets.

Add Helm repository:

```
# Add External Secrets Operator Helm repo
helm repo add external-secrets https://charts.external-secrets.io
helm repo update

# Verify repo added
helm repo list

# Expected output:
# NAME                                URL
# external-secrets                    https://charts.external-secrets.io
```

Install External Secrets Operator:

```
# Create namespace
kubectl create namespace external-secrets-system

# Install operator
helm install external-secrets \
  external-secrets/external-secrets \
  -n external-secrets-system \
  --set installCRDs=true

# Expected output:
# NAME: external-secrets
# NAMESPACE: external-secrets-system
# STATUS: deployed
# ...
```

Verify installation:

```
# Check Pods are running
kubectl get pods -n external-secrets-system

# Expected output:
# NAME                                READY   STATUS
# external-secrets-operator-xxx      1/1     Running
# external-secrets-cert-controller-xxx 1/1     Running

# Check CRDs installed
kubectl get crd | grep external-secrets

# Expected: secretstores.external-secrets.io,
externalsecrets.external-secrets.io
```

Common Installation Issues:**Issue 1: "Error: INSTALLATION FAILED"**

```
# Check if namespace exists
kubectl get namespace external-secrets-system

# If not, create it
kubectl create namespace external-secrets-system

# Retry installation
helm install external-secrets external-secrets/external-secrets -n
external-secrets-system --set installCRDs=true
```

Issue 2: "Pods stuck in Pending"

```
# Check Pod status
kubectl describe pod -n external-secrets-system
```



```
# Common causes:
# - Insufficient resources (check node capacity)
# - Image pull errors (check network connectivity)
```

Checkpoint - Operator Installed:

- ☐ Helm repo added
- ☐ External Secrets Operator installed
- ☐ Pods running
- ☐ CRDs installed

External Secrets Operator installed!

Step 3: Configure IRSA for External Secrets

What we're doing: Setting up IRSA so External Secrets Operator can read secrets from AWS without access keys.

Note: IRSA is covered in detail in D4. This step provides a preview. You'll understand it fully in D4.

Check if OIDC provider exists:

```
# Get EKS cluster name
EKS_CLUSTER=$(aws eks list-clusters --query 'clusters[0]' --output
text)

# Get OIDC provider URL
aws eks describe-cluster --name $EKS_CLUSTER --query
'cluster.identity.oidc.issuer' --output text

# Expected output:
#
https://oidc.eks.us-east-1.amazonaws.com/id/EXAMPLED539D4633E53DE1B71
6D3041E
```

Create IAM role for External Secrets:

```
# File: irsa-external-secrets.tf
# =====
# IRSA: IAM Role for External Secrets Operator
# =====

# Use existing OIDC provider (already created in iam.tf)
data "aws_eks_cluster" "main" {
  name = var.eks_cluster_name # Use variable, not hardcoded
}

# Reference existing OIDC provider (already in iam.tf)
data "aws_iam_openid_connect_provider" "eks" {
  url = aws_eks_cluster.main.identity[0].oidc[0].issuer
}

# IAM role that External Secrets Operator can assume
resource "aws_iam_role" "external_secrets" {
  name = "${local.name_prefix}-external-secrets-role"

  assume_role_policy = jsonencode({
    Version = "2012-10-17"
    Statement = [{
      Effect = "Allow"
      Principal = {
        Federated = data.aws_iam_openid_connect_provider.eks.arn
      }
      Action = "sts:AssumeRoleWithWebIdentity"
      Condition = {
        StringEquals = {
          "${replace(data.aws_iam_openid_connect_provider.eks.url,
"https://", "")}:sub" =
"system:serviceaccount:external-secrets-system:external-secrets"
          "${replace(data.aws_iam_openid_connect_provider.eks.url,
"https://", "")}:aud" = "sts.amazonaws.com"
        }
      }
    }]
  })

  tags = merge(
```

```

    local.common_tags,
    {
      Name = "${local.name_prefix}-external-secrets-role"
      Task = "D3"
    }
  )
}

# IAM policy: Allow reading secrets from Secrets Manager
resource "aws_iam_role_policy" "external_secrets" {
  name = "secrets-manager-access"
  role = aws_iam_role.external_secrets.id

  policy = jsonencode({
    Version = "2012-10-17"
    Statement = [{
      Effect = "Allow"
      Action = [
        "secretsmanager:GetSecretValue",
        "secretsmanager:DescribeSecret"
      ]
      Resource = [
        aws_secretsmanager_secret.rds_password.arn, # Use existing secret
        # from Terraform
        "arn:aws:secretsmanager:${var.aws_region}::secret:h-game/*" #
        Allow any h-game secrets
      ]
    }, {
      Effect = "Allow"
      Action = [
        "kms:Decrypt"
      ]
      Resource = "*"
      Condition = {
        StringEquals = {
          "kms:ViaService" =
            "secretsmanager.${var.aws_region}.amazonaws.com"
        }
      }
    }
  ]
})
}

```

```
# Output: IAM role ARN (needed for Service Account)
output "external_secrets_role_arn" {
  description = "ARN of IAM role for External Secrets Operator"
  value       = aws_iam_role.external_secrets.arn
}
```

Apply Terraform changes:

```
# Preview changes
terraform plan

# Should show:
# + aws_iam_role.external_secrets
# + aws_iam_role_policy.external_secrets

# Apply changes
terraform apply

# Type 'yes' when prompted

# Get IAM role ARN
terraform output external_secrets_role_arn
```

Create Service Account with IAM annotation:

```
# Get IAM role ARN from Terraform output
ROLE_ARN=$(terraform output -raw external_secrets_role_arn)

# Create Service Account
cat > external-secrets-sa.yaml << EOF
apiVersion: v1
kind: ServiceAccount
metadata:
  name: external-secrets
  namespace: external-secrets-system
  annotations:
```

```
eks.amazonaws.com/role-arn: ${ROLE_ARN}
EOF

# Apply Service Account
kubectl apply -f external-secrets-sa.yaml

# Verify Service Account
kubectl get serviceaccount external-secrets -n
external-secrets-system
```

Update External Secrets Operator to use Service Account:

```
# Update Helm release to use Service Account
helm upgrade external-secrets \
  external-secrets/external-secrets \
  -n external-secrets-system \
  --set serviceAccount.name=external-secrets \
  --set serviceAccount.create=false \
  --reuse-values

# Restart operator Pods to pick up new Service Account
kubectl rollout restart deployment external-secrets-operator -n
external-secrets-system

# Wait for Pods to restart
kubectl get pods -n external-secrets-system -w
```

Checkpoint - IRSA Configured:

- IAM role created
- Service Account created with annotation
- External Secrets Operator using Service Account
- Pods restarted

IRSA configured!

Step 4: Create SecretStore

What we're doing: Creating a SecretStore that tells External Secrets Operator how to connect to AWS Secrets Manager.

Create SecretStore:

```
# Navigate to k8s directory
cd ~/projects/H-Game/k8s

# Create SecretStore
cat > secretstore.yaml << 'EOF'
apiVersion: external-secrets.io/v1beta1
kind: SecretStore
metadata:
  name: aws-secretsmanager
  namespace: h-game
spec:
  provider:
    aws:
      service: SecretsManager
      region: us-east-1
      auth:
        jwt:
          serviceAccountRef:
            name: external-secrets
            namespace: external-secrets-system
EOF
```

```
# Apply SecretStore
kubectl apply -f secretstore.yaml

# Verify SecretStore
kubectl get secretstore -n h-game

# Expected output:
# NAME                                AGE    STATUS
# aws-secretsmanager                 10s    Valid
```

Verify SecretStore status:

```
# Check SecretStore details
kubectl describe secretstore aws-secretsmanager -n h-game

# Expected output:
# Status:
# Conditions:
#   Last Transition Time: 2024-01-15T10:30:00Z
#   Message:             SecretStore validated successfully
#   Reason:              Valid
#   Status:              True
#   Type:                Ready
```

Common Issues:**Issue 1: "SecretStore not ready"**

```
# Check SecretStore status
kubectl describe secretstore aws-secretsmanager -n h-game

# Common causes:
# - IRSA not configured (check Service Account annotation)
# - IAM role doesn't have permissions (check IAM policy)
# - Wrong region (check SecretStore region matches secret region)
```

Issue 2: "Authentication failed"

```
# Verify Service Account has IAM role annotation
kubectl get serviceaccount external-secrets -n
external-secrets-system -o yaml

# Should show:
# annotations:
#   eks.amazonaws.com/role-arn:
arn:aws:iam::123456789012:role/h-game-external-secrets
```

Checkpoint - SecretStore Created:

- SecretStore created
- Status: Valid
- Ready for ExternalSecret

SecretStore ready!

Step 5: Create ExternalSecret

What we're doing: Creating an ExternalSecret that syncs AWS Secrets Manager secret to Kubernetes Secret.

Create ExternalSecret:

```
# Create ExternalSecret
cat > external-secret.yaml << 'EOF'
apiVersion: external-secrets.io/v1beta1
kind: ExternalSecret
metadata:
  name: database-credentials
  namespace: h-game
spec:
  refreshInterval: 1h # Sync every hour
  secretStoreRef:
    name: aws-secretsmanager
    kind: SecretStore
  target:
    name: postgres-secret # Kubernetes Secret name (match your deployment)
    creationPolicy: Owner
  data:
    # Map to postgres-user (match backend-deployment.yaml)
    - secretKey: postgres-user
      remoteRef:
        key: h-game/rds/password # Use existing Terraform secret
        property: username
    # Map to postgres-password (match backend-deployment.yaml)
    - secretKey: postgres-password
      remoteRef:
        key: h-game/rds/password
        property: password
```



```
# Map to postgres-db (match backend-deployment.yaml)
- secretKey: postgres-db
  remoteRef:
    key: h-game/rds/password
    property: dbname
EOF
```

```
# Apply ExternalSecret
kubectl apply -f external-secret.yaml

# Verify ExternalSecret
kubectl get externalsecret -n h-game

# Expected output:
# NAME                                STORE                                REFRESH INTERVAL
STATUS
# database-credentials  aws-secretsmanager                  1h
SecretSynced
```

Verify Kubernetes Secret created:

```
# Check if Kubernetes Secret was created
kubectl get secret postgres-secret -n h-game

# Expected output:
# NAME            TYPE      DATA   AGE
# postgres-secret  Opaque    2       30s

# Verify secret values (should match AWS Secrets Manager)
kubectl get secret postgres-secret -n h-game -o
jsonpath='{.data.username}' | base64 -d
echo ""

# Expected: gameuser

kubectl get secret postgres-secret -n h-game -o
jsonpath='{.data.password}' | base64 -d
```

```
echo ""  
  
# Expected: <password from AWS Secrets Manager>
```

Check ExternalSecret status:

```
# View ExternalSecret details  
kubectl describe externalsecret database-credentials -n h-game  
  
# Expected output:  
# Status:  
# Conditions:  
#   Last Transition Time: 2024-01-15T10:30:00Z  
#   Message:             Secret synced successfully  
#   Reason:              SecretSynced  
#   Status:              True  
#   Type:               Ready  
#   Secret Hash:         abc123def456
```

Checkpoint - ExternalSecret Created:

- ExternalSecret created
- Status: SecretSynced
- Kubernetes Secret created
- Values match AWS Secrets Manager

ExternalSecret working!

Step 6: Update Deployment

What we're doing: Updating your backend Deployment to use the synced secret (no code changes needed).

Verify current Deployment:

```
# Check current backend Deployment
kubectl get deployment backend -n h-game -o yaml | grep -A 10
secretKeyRef

# Should show existing secretKeyRef to postgres-secret
```

Update Deployment (if needed):

```
# Your Deployment should already reference postgres-secret
# If it does, no changes needed!
# External Secrets Operator updates the same Secret

# If you need to update, edit deployment
kubectl edit deployment backend -n h-game

# Ensure env vars reference postgres-secret:
# env:
# - name: DB_USER
#   valueFrom:
#     secretKeyRef:
#       name: postgres-secret
#       key: username
# - name: DB_PASSWORD
#   valueFrom:
#     secretKeyRef:
#       name: postgres-secret
#       key: password
```

Restart Pods to pick up new secret:

```
# Restart backend Pods
kubectl rollout restart deployment backend -n h-game

# Watch rollout
kubectl rollout status deployment backend -n h-game

# Expected: deployment "backend" successfully rolled out
```

Verify Pods are using secret:

```
# Check Pod logs for database connection
kubectl logs -l app=backend -n h-game --tail=20 | grep -i database

# Expected output:
# Connecting to database...
# Database connected successfully!
```

Checkpoint - Deployment Updated:

- ☐ Deployment references postgres-secret
- ☐ Pods restarted
- ☐ Database connection working
- ☐ Using secrets from AWS Secrets Manager

Deployment updated!

Step 7: Enable Auto-Rotation

What we're doing: Configuring automatic secret rotation every 30 days.

Create rotation Lambda function (simplified):

Note: Full Lambda rotation function is complex. For learning, we'll configure rotation but use a simple template. Production would use a custom Lambda.

 **IMPORTANT:** The managed Lambda function may not exist in your account. Check first:

```
# Check if rotation Lambda exists
aws lambda get-function \
  --function-name SecretsManagerRDSPostgreSQLRotationSingleUser \
  --region us-east-1 2>&1

# If it doesn't exist, you'll need to create it or use a custom
Lambda
```

Option A: Use Managed Lambda (if it exists):

```
# Enable rotation for database secret
aws secretsmanager rotate-secret \
  --secret-id h-game/rds/password \
  --rotation-lambda-arn arn:aws:lambda:us-east-1:$(aws sts
get-caller-identity --query Account --output
text):function:SecretsManagerRDSPostgreSQLRotationSingleUser \
  --rotation-rules AutomaticallyAfterDays=30 \
  --region us-east-1
```

Option B: Terraform Configuration (add to `secrets-manager.tf`):

```
# =====
# SECRET ROTATION CONFIGURATION
# =====
#  NOTE: This requires the rotation Lambda function to exist
# For learning, rotation is configured but may not execute without
Lambda
```

```

resource "aws_secretsmanager_secret_rotation" "rds_password" {
  secret_id = aws_secretsmanager_secret.rds_password.id

  # ⚠️ Replace with actual Lambda ARN if it exists
  # Check: aws lambda list-functions --query
  'Functions[?contains(FunctionName, `Rotation`)].FunctionName'
  rotation_lambda_arn =
  "arn:aws:lambda:${var.aws_region}:${data.aws_caller_identity.current.
  account_id}:function:SecretsManagerRDSPostgreSQLRotationSingleUser"

  rotation_rules {
    automatically_after_days = 30
  }

  # Only create if Lambda exists
  lifecycle {
    ignore_changes = [rotation_lambda_arn]
  }
}

```

Verify rotation configured:

```

# Check rotation status
aws secretsmanager describe-secret \
  --secret-id h-game/rds/password \
  --region us-east-1 \
  --query 'RotationEnabled'

# Expected: true (if Lambda exists and rotation enabled)

```

Checkpoint - Rotation Enabled:

- Rotation configured
- Schedule: Every 30 days
- External Secrets Operator will sync new values

Auto rotation enabled!

Step 8: Verify & Test

What we're doing: Testing the complete secrets management workflow.

Test 1: Verify secret sync

```
# Update secret in AWS Secrets Manager
NEW_PASSWORD=$(openssl rand -base64 32)
aws secretsmanager update-secret \
  --secret-id h-game/rds/password \
  --secret-string "$(aws secretsmanager get-secret-value --secret-id \
h-game/rds/password --region us-east-1 --query SecretString --output \
text | jq ".password = \"${NEW_PASSWORD}\"")" \
  --region us-east-1

# Wait for External Secrets Operator to sync (up to 1 hour, or
trigger manually)
# Check ExternalSecret status
kubectl get externalsecret database-credentials -n h-game

# Should show: SecretSynced
```

Test 2: Verify Pods get updated secret

```
# After sync, check if Pods have new secret
# Note: Pods need to be restarted to pick up new secret values
kubectl rollout restart deployment backend -n h-game

# Verify new password is being used
kubectl logs -l app=backend -n h-game --tail=20 | grep -i database
```

Test 3: Verify no hardcoded secrets

```
# Search codebase for hardcoded secrets
cd ~/projects/H-Game
grep -r "password.*=" backend/ --exclude-dir=node_modules || echo "No
hardcoded passwords found"
```

```
# Search for API keys
grep -r "api.*key.*=" backend/ --exclude-dir=node_modules || echo "No
hardcoded API keys found"
```

Test 4: Verify audit logging

```
# Check CloudTrail for secret access
aws cloudtrail lookup-events \
  --lookup-attributes
  AttributeKey=EventName,AttributeValue=GetSecretValue \
  --max-results 10 \
  --region us-east-1

# Expected: Shows who accessed secrets and when
```

Checkpoint - Complete Workflow Verified:

- ☐ Secret sync working
- ☐ Pods using secrets from AWS
- ☐ No hardcoded secrets in code
- ☐ Audit logging working

Secrets management complete!

Step 9: Commit Your Work

What we're doing: Committing secrets management configuration to Git.

```
# Navigate to project root
cd ~/projects/H-Game

# Add Kubernetes manifests
git add k8s/secretstore.yaml
git add k8s/external-secret.yaml
git add k8s/external-secrets-sa.yaml

# Navigate to Terraform directory
cd ~/projects/h-game-terraform

# Add Terraform files
git add irsa-external-secrets.tf
git add secrets-manager.tf

# Commit with clear message
cd ~/projects/H-Game
git commit -m "feat(security): Add AWS Secrets Manager integration

- Store secrets in AWS Secrets Manager (encrypted)
- Install External Secrets Operator
- Configure IRSA for secure secret access
- Sync secrets to Kubernetes automatically
- Enable auto-rotation (30 days)

Closes: DEVOPS-6103"
```

```
# Push to remote
git push origin main
```

Checkpoint - Work Committed:

- Kubernetes manifests committed
- Terraform files committed
- Clear commit message
- Changes pushed to remote

Work committed!**Phase 3: Validation & Consolidation (20 min)****Quiz:** Secrets Management Understanding**Answer these questions to verify your understanding:**

Question 1: Why use AWS Secrets Manager instead of Kubernetes Secrets?

Answer:

- **AWS Secrets Manager:** Encrypted at rest, auto-rotation, audit logs, IAM-controlled
- **Kubernetes Secrets:** Base64-encoded (not encrypted), no auto-rotation, stored in etcd

Key point: AWS Secrets Manager is production-grade; Kubernetes Secrets are for development.

Question 2: What is External Secrets Operator?

Answer:

- Syncs AWS Secrets Manager → Kubernetes Secrets automatically
- Pods still use Kubernetes Secrets (no code changes)
- Secrets update automatically when rotated in AWS

Key point: Bridge between AWS Secrets Manager and Kubernetes.

Knowledge Check

Test your understanding before moving on:

1. Where are secrets stored?

- ☐ a) Kubernetes only
- ☐ b) AWS Secrets Manager
- ☐ c) Git repository

2. What syncs secrets to Kubernetes?

- ☐ a) kubectl
- ☐ b) External Secrets Operator
- ☐ c) Terraform

3. How often should secrets rotate?

- ☐ a) Never
- ☐ b) Every 30-90 days
- ☐ c) Every day

4. What is IRSA?

- ☐ a) A secret storage system
- ☐ b) IAM Roles for Service Accounts
- ☐ c) A Kubernetes Secret type

5. Are Kubernetes Secrets encrypted?

- ☐ a) Yes, always
- ☐ b) No, only base64-encoded
- ☐ c) Only in production

Score:

- 5/5: Excellent! Move to D4
- 3-4/5: Review weak areas
- <3/5: Re-read Phase 1

Explanations

1. Secrets stored in AWS Secrets Manager (encrypted, auto-rotated)
2. External Secrets Operator syncs AWS → Kubernetes automatically
3. Rotate every 30-90 days (reduces risk if compromised)
4. IRSA = IAM Roles for Service Accounts (Pods use IAM roles, not access keys)
5. Kubernetes Secrets are base64-encoded, NOT encrypted (AWS Secrets Manager is encrypted)

Checklist: Did You Complete Everything?

Before moving on, check these boxes:

Understanding Phase

- ☐ I understand why AWS Secrets Manager is better than Kubernetes Secrets
- ☐ I know what External Secrets Operator does
- ☐ I understand secret rotation
- ☐ I can explain IRSA (preview)

Implementation Phase

- ☐ Secrets created in AWS Secrets Manager
- ☐ External Secrets Operator installed
- ☐ IRSA configured (preview)
- ☐ SecretStore created
- ☐ ExternalSecret created
- ☐ Deployment updated
- ☐ Auto-rotation enabled
- ☐ Workflow tested end-to-end

Verification Phase

- ☐ Can explain secrets management flow
- ☐ Can view secrets in AWS
- ☐ Can verify secrets synced to Kubernetes
- ☐ Can verify no hardcoded secrets

Git

- ☐ Committed Kubernetes manifests
- ☐ Committed Terraform files
- ☐ Documented secrets management

Key Takeaways

The Big Picture

What you learned in this task:

- Secrets stored in AWS Secrets Manager (encrypted)
- External Secrets Operator syncing to Kubernetes
- Auto-rotation configured (30 days)
- IRSA preview (full coverage in D4)
- No hardcoded secrets in code

Secrets are now managed securely!

Critical Concepts

1. Secrets Management:

- AWS Secrets Manager: Encrypted storage, auto-rotation, audit logs
- External Secrets Operator: Syncs AWS → Kubernetes
- IRSA: Pods use IAM roles (no access keys)

2. Secret Flow:

- Store: AWS Secrets Manager
- Sync: External Secrets Operator
- Use: Kubernetes Secrets (Pods read from here)
- Rotate: AWS rotates automatically

3. Security Benefits:

- Encrypted at rest
- Auto-rotation reduces risk
- Audit logs track access
- IAM-controlled access

4. Automation:

- External Secrets Operator syncs automatically
- Rotation happens automatically
- No manual intervention needed

What's Next?

Congratulations! You've secured your secrets!

You now have:

- Secrets in AWS Secrets Manager
- Auto-rotation configured
- External Secrets Operator syncing
- IRSA preview (full coverage next)

Next: TASK D4 - IAM & Zero Trust

In D4, you'll learn:

- How IRSA works in detail
- How to implement least-privilege IAM policies
- How to use network policies for zero trust
- How all security layers work together

But most importantly: You now understand secrets management. Combined with scanning, you have secure credential handling!

Proceed to TASK D4: IAM & Zero Trust

TASK D4: IAM & Zero Trust

Time: 3 hours | **Priority:** High | **Ticket:** DEVOPS-6104 | **Cost:** Free

Mission Brief

Implement IRSA (IAM Roles for Service Accounts) so Pods authenticate with IAM roles instead of access keys. Apply least-privilege policies and network policies for zero-trust security.

Outcomes:

- Understand IRSA and why it matters
- Configure IAM roles for Pods
- Implement least-privilege IAM policies
- Create network policies for zero trust
- Eliminate access keys from Pods

Prerequisites: Complete D3 (Secrets Management) - IRSA preview

Setup Note: IRSA uses IAM roles, not access keys. Network policies restrict Pod-to-Pod traffic.

Cost Note: This task costs \$0. IAM roles are free. Network policies are free.

The One Thing That Makes This Click

IRSA & Zero Trust

IRSA (IAM Roles for Service Accounts) lets Kubernetes Pods access AWS **without using access keys**.

Instead of putting AWS access keys inside Pods, each Pod uses an IAM role with only the permissions it needs.

Without IRSA

- Pods store **AWS access keys** in Kubernetes Secrets
- The same key is often shared by many Pods
- Keys do not expire unless rotated manually
- If one Pod is compromised, everything is exposed

This is risky and hard to manage.

With IRSA

- Pods authenticate using IAM roles
- Each service gets its own role
- Permissions follow least privilege
- Credentials are temporary and auto-expire

Key rule: Pods use IAM roles, not access keys.

How IRSA Works (High Level)

1. A Kubernetes Service Account is linked to an IAM role
2. A Pod runs using that Service Account
3. AWS verifies the identity using OIDC
4. AWS issues temporary credentials (valid for ~1 hour)
5. The Pod accesses AWS services securely
6. Credentials expire and refresh automatically

No access keys are created or stored anywhere.

Zero Trust (Why It Matters)

Zero Trust means: “Never trust by default. Allow only what is required.”

Traditional (Weak)

- All Pods can talk to each other
- Broad IAM permissions
- One compromised Pod can access everything

Zero Trust (Strong)

- Each Pod has **minimum IAM permissions**
- **Network Policies** restrict Pod-to-Pod traffic
- If one Pod is compromised, damage is limited

Example:

- Backend Pod, allowed to talk to **PostgreSQL (5432)** and **Redis (6379)**
- Everything else, blocked by default

What Changes After This Task

Before

- Pods use access keys (or none)
- Permissive IAM policies
- No network restrictions

After

- Pods use **IRSA**
- **Least-privilege IAM roles**
- **Network Policies** enforce Zero Trust
- **Zero access keys** in Kubernetes

Takeaway

IRSA removes access keys from Pods, and Zero Trust ensures every Pod can only do what it truly needs, nothing more.

We'll complete this in 7 steps:

1. Verify OIDC provider (*10 min*)
2. Create IAM roles for each service (*30 min*)
3. Create Service Accounts and link them to IAM roles (*20 min*)
4. Update Deployments to use Service Accounts (*20 min*)
5. Create Network Policies to restrict Pod traffic (*30 min*)
6. Remove AWS access keys from Kubernetes (*20 min*)
7. Verify and test IRSA and Network Policies (*20 min*)

Step 1: Verify OIDC Provider

What we're doing: Checking if EKS cluster has OIDC provider configured (required for IRSA).

Check OIDC provider:

```
# Get EKS cluster name from Terraform
cd ~/projects/h-game-terraform
EKS_CLUSTER=$(terraform output -raw eks_cluster_name)

# Get OIDC issuer URL
OIDC_URL=$(aws eks describe-cluster --name $EKS_CLUSTER --query
'cluster.identity.oidc.issuer' --output text)
echo "OIDC URL: $OIDC_URL"

# Check if OIDC provider exists in IAM (should already exist from
Terraform)
aws iam list-open-id-connect-providers

# Should show provider with ARN containing the OIDC ID
```

Check if OIDC provider exists in IAM:

```
# Extract OIDC ID from URL
OIDC_ID=$(echo $OIDC_URL | cut -d '/' -f 5)

# Check if provider exists
aws iam list-open-id-connect-providers

# Should show provider with ARN containing the OIDC ID
```

NO NEED TO CREATE OIDC PROVIDER - It's already in `iam.tf`!

Checkpoint - OIDC Provider Ready:

- ☐ OIDC provider exists or created
- ☐ EKS cluster has OIDC issuer
- ☐ Ready for IRSA

OIDC provider ready!

Step 2: Create IAM Roles

What we're doing: Creating IAM roles for each service that needs AWS access.

Create IAM role for External Secrets Operator:

```
# Navigate to Terraform directory
cd ~/projects/h-game-terraform

# File: irsa-external-secrets.tf
# =====
# IRSA: IAM Role for External Secrets Operator
# =====
# NOTE: This may already exist from D3 - check first!

# Use existing OIDC provider (already created in iam.tf)
data "aws_eks_cluster" "main" {
  name = var.eks_cluster_name # Use variable, not hardcoded
}

# Reference existing OIDC provider (already in iam.tf)
data "aws_iam_openid_connect_provider" "eks" {
  url = aws_eks_cluster.main.identity[0].oidc[0].issuer
}

resource "aws_iam_role" "external_secrets" {
  name = "${local.name_prefix}-external-secrets-role"
```

```

assume_role_policy = jsonencode({
  Version = "2012-10-17"
  Statement = [{
    Effect = "Allow"
    Principal = {
      Federated = data.aws_iam_openid_connect_provider.eks.arn
    }
    Action = "sts:AssumeRoleWithWebIdentity"
    Condition = {
      StringEquals = {
        "${replace(data.aws_iam_openid_connect_provider.eks.url,
"https://", "")}:sub" =
"system:serviceaccount:external-secrets-system:external-secrets"
        "${replace(data.aws_iam_openid_connect_provider.eks.url,
"https://", "")}:aud" = "sts.amazonaws.com" # ← ADDED: Required for
IRSA
      }
    }
  }]
})

tags = merge(
  local.common_tags,
  {
    Name = "${local.name_prefix}-external-secrets-role"
    Task = "D4"
  }
)
}

resource "aws_iam_role_policy" "external_secrets" {
  name = "secrets-manager-access"
  role = aws_iam_role.external_secrets.id

  policy = jsonencode({
    Version = "2012-10-17"
    Statement = [{
      Effect = "Allow"

```

```

    Action = [
        "secretsmanager:GetSecretValue",
        "secretsmanager:DescribeSecret"
    ]
    Resource = [
        aws_secretsmanager_secret.rds_password.arn, # Reference
existing secret
        "arn:aws:secretsmanager:${var.aws_region}::secret:h-game/*"
    ]
}, {
    Effect = "Allow"
    Action = [
        "kms:Decrypt"
    ]
    Resource = "*"
    Condition = {
        StringEquals = {
            "kms:ViaService" =
"secretsmanager.${var.aws_region}.amazonaws.com"
        }
    }
}]
})
}

output "external_secrets_role_arn" {
    description = "ARN of IAM role for External Secrets Operator"
    value       = aws_iam_role.external_secrets.arn
}

```

Create IAM role for backend (if it needs AWS access):

```

# Create IAM role for backend
cat > irsa-backend.tf << 'EOF'
# File: irsa-backend.tf
# =====
# IRSA: IAM Role for Backend Pods
# =====

```

```

# Only create if backend needs AWS access (S3, SQS, etc.)
# If backend only uses RDS/ElastiCache, you may not need this

data "aws_eks_cluster" "main" {
  name = var.eks_cluster_name # Use variable
}

data "aws_iam_openid_connect_provider" "eks" {
  url = aws_eks_cluster.main.identity[0].oidc[0].issuer
}

resource "aws_iam_role" "backend" {
  name = "${local.name_prefix}-backend-role"

  assume_role_policy = jsonencode({
    Version = "2012-10-17"
    Statement = [{
      Effect = "Allow"
      Principal = {
        Federated = data.aws_iam_openid_connect_provider.eks.arn
      }
      Action = "sts:AssumeRoleWithWebIdentity"
      Condition = {
        StringEquals = {
          "${replace(data.aws_iam_openid_connect_provider.eks.url,
"https://", "")}:sub" = "system:serviceaccount:h-game:backend"
          "${replace(data.aws_iam_openid_connect_provider.eks.url,
"https://", "")}:aud" = "sts.amazonaws.com" # ← ADDED
        }
      }
    }]
  })

  tags = merge(
    local.common_tags,
    {
      Name = "${local.name_prefix}-backend-role"
      Task = "D4"
    }
  )

```

```

    }
  )
}

# Example: If backend needs S3 access
resource "aws_iam_role_policy" "backend_s3" {
  name = "s3-read-access"
  role = aws_iam_role.backend.id

  policy = jsonencode({
    Version = "2012-10-17"
    Statement = [{
      Effect = "Allow"
      Action = [
        "s3:GetObject",
        "s3:ListBucket"
      ]
      Resource = [
        "arn:aws:s3:::h-game-*",
        "arn:aws:s3:::h-game-*/*"
      ]
    }]
  })
}

output "backend_role_arn" {
  description = "ARN of IAM role for Backend Pods"
  value       = aws_iam_role.backend.arn
}

```

Apply Terraform:

```

# Preview changes
terraform plan

# Apply changes
terraform apply

```



```
# Type 'yes' when prompted

# Get IAM role ARNs
terraform output external_secrets_role_arn
terraform output backend_role_arn # If created
```

Checkpoint - IAM Roles Created:

- External Secrets role created
- Backend role created (if needed)
- Roles have correct assume role policy
- Policies attached

Step 3: Create Service Accounts

What we're doing: Creating Service Accounts with IAM role annotations.

Update External Secrets Service Account:

```
# Get IAM role ARN
ROLE_ARN=$(terraform output -raw external_secrets_role_arn)

# Update Service Account (from D3)
cat > external-secrets-sa.yaml << EOF
apiVersion: v1
kind: ServiceAccount
metadata:
  name: external-secrets
  namespace: external-secrets-system
  annotations:
    eks.amazonaws.com/role-arn: ${ROLE_ARN}
EOF
```

```
# Apply Service Account
kubectl apply -f external-secrets-sa.yaml

# Verify annotation
kubectl get serviceaccount external-secrets -n
```

```
external-secrets-system -o yaml | grep role-arn
```

Create backend Service Account:

```
# Get backend IAM role ARN (if created)
BACKEND_ROLE_ARN=$(terraform output -raw backend_role_arn 2>/dev/null
|| echo "")

# Create backend Service Account
cat > backend-sa.yaml << EOF
apiVersion: v1
kind: ServiceAccount
metadata:
  name: backend
  namespace: h-game
  annotations:
    eks.amazonaws.com/role-arn: ${BACKEND_ROLE_ARN}
EOF

# Apply Service Account
kubectl apply -f backend-sa.yaml

# Verify
kubectl get serviceaccount backend -n h-game
```

Checkpoint - Service Accounts Created:

- External Secrets Service Account updated
- Backend Service Account created
- IAM role annotations set
- Ready for Pods to use

Service Accounts ready!

Step 4: Update Deployments

What we're doing: Updating Deployments to use Service Accounts.

Update External Secrets Operator:

```
# Update Helm release to use Service Account
helm upgrade external-secrets \
  external-secrets/external-secrets \
  -n external-secrets-system \
  --set serviceAccount.name=external-secrets \
  --set serviceAccount.create=false \
  --reuse-values

# Restart operator
kubectl rollout restart deployment external-secrets-operator -n
external-secrets-system
```

Update backend Deployment:

```
# Edit backend Deployment
kubectl edit deployment backend -n h-game

# Add serviceAccountName:
# spec:
#   serviceAccountName: backend # Add this line
#   containers:
#     - name: backend
#       ...

# Or create updated Deployment file
cat > backend-deployment.yaml << 'EOF'
apiVersion: apps/v1
kind: Deployment
metadata:
  name: backend
  namespace: h-game
spec:
  replicas: 3
```

```

selector:
  matchLabels:
    app: backend
template:
  metadata:
    labels:
      app: backend
  spec:
    serviceAccountName: backend # ← Add this
    containers:
      - name: backend
        # ... rest of config ...
EOF

```

```

# Apply updated Deployment
kubectl apply -f backend-deployment.yaml

# Watch rollout
kubectl rollout status deployment backend -n h-game

```

Verify Pods are using Service Account:

```

# Check Pod spec
kubectl get pod -l app=backend -n h-game -o yaml | grep
serviceAccountName

# Should show: serviceAccountName: backend

# Check Pod can assume IAM role
kubectl exec -it $(kubectl get pod -l app=backend -n h-game -o
jsonpath='{.items[0].metadata.name}') -n h-game -- env | grep AWS

# Should show AWS environment variables (if Pod needs AWS access)

```

Checkpoint - Deployments Updated:

- External Secrets Operator using Service Account
- Backend Deployment using Service Account
- Pods restarted
- Service Accounts in use

Deployments updated!

Step 5: Create Network Policies

What we're doing: Creating network policies to restrict Pod-to-Pod traffic (zero trust).

Create network policy for backend:

```
# Navigate to k8s directory
cd ~/projects/H-Game/k8s

# Create network policy
cat > network-policy-backend.yaml << 'EOF'
# network-policy-backend.yaml
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: backend-network-policy
  namespace: h-game
spec:
  podSelector:
    matchLabels:
      app: backend
  policyTypes:
    - Ingress
    - Egress

# Ingress: Allow traffic from frontend only
ingress:
  - from:
      - podSelector:
          matchLabels:
            app: frontend
    ports:
      - protocol: TCP
        port: 3001

# Egress: Allow traffic to external services and DNS
egress:
  # Allow DNS (REQUIRED for service discovery and external DNS)
```

```

- to:
  - namespaceSelector: {}
  ports:
    - protocol: UDP
      port: 53
    - protocol: TCP
      port: 53

# Allow HTTPS to AWS services (RDS, ElastiCache, Secrets Manager,
etc.)
- to: [] # Empty = all destinations
  ports:
    - protocol: TCP
      port: 443 # HTTPS for AWS API calls

# Allow PostgreSQL to RDS (external)
- to: [] # RDS is outside cluster, so allow all egress
  ports:
    - protocol: TCP
      port: 5432

# Allow Redis to ElastiCache (external)
- to: [] # ElastiCache is outside cluster
  ports:
    - protocol: TCP
      port: 6379

# Allow HTTP/HTTPS for external API calls (if needed)
- to: []
  ports:
    - protocol: TCP
      port: 80
    - protocol: TCP
      port: 443

EOF

# Apply network policy

```

```
kubectl apply -f network-policy-backend.yaml
```

```
# Verify network policy
```

```
kubectl get networkpolicy -n h-game
```

```
# Expected output:
```

```
# NAME                                POD-SELECTOR  AGE
# backend-network-policy  app=backend   10s
```

⚠ IMPORTANT NOTE: Since RDS and ElastiCache are outside the Kubernetes cluster, network policies can't restrict traffic to them directly. The policy above allows egress to external services. For true zero-trust with external services, you'd need:

- Security Groups (which we have already configured in Terraform)
- VPC endpoints
- Service mesh (Istio/Linkerd)

Alternative: Network Policy for Frontend-to-Backend Only:

```
# network-policy-backend.yaml (simplified - only Pod-to-Pod)
```

```
apiVersion: networking.k8s.io/v1
```

```
kind: NetworkPolicy
```

```
metadata:
```

```
  name: backend-network-policy
```

```
  namespace: h-game
```

```
spec:
```

```
  podSelector:
```

```
    matchLabels:
```

```
      app: backend
```

```
  policyTypes:
```

```
  - Ingress
```

```
  - Egress
```

```
# Ingress: Only from frontend
```

```
ingress:
```

```
- from:
```

```
  - podSelector:
```

```
    matchLabels:
```

```
      app: frontend
```



```

ports:
- protocol: TCP
  port: 3001

# Egress: Allow all (since RDS/ElastiCache are external)
# Network policies can't restrict external traffic effectively
egress:
- {} # Allow all egress (external services need this)

```

Network Policy for Frontend:

```

# network-policy-frontend.yaml
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: frontend-network-policy
  namespace: h-game
spec:
  podSelector:
    matchLabels:
      app: frontend
  policyTypes:
  - Ingress
  - Egress

# Ingress: Allow from ALB/Ingress (external)
ingress:
- {} # Allow all ingress (from ALB)

# Egress: Only to backend
egress:
- to:
  - podSelector:
      matchLabels:
        app: backend
  ports:
  - protocol: TCP
    port: 3001

```

```
# Allow DNS
- to:
  - namespaceSelector: {}
  ports:
    - protocol: UDP
      port: 53
    - protocol: TCP
      port: 53
```

Apply Network Policies:

```
cd ~/projects/H-Game/k8s
kubectl apply -f network-policy-backend.yaml
kubectl apply -f network-policy-frontend.yaml

# Verify
kubectl get networkpolicy -n h-game
```

⚠ NOTE: If you still have a local PostgreSQL pod (not just RDS), you can add a policy for it:

```
# network-policy-postgres.yaml (only if you have local postgres pod)
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: postgres-network-policy
  namespace: h-game
spec:
  podSelector:
    matchLabels:
      app: postgres
  policyTypes:
    - Ingress
    - Egress

# Ingress: Only from backend (if using local postgres)
ingress:
```

```
- from:
  - podSelector:
      matchLabels:
        app: backend
  ports:
    - protocol: TCP
      port: 5432

# Egress: DNS only
egress:
- to:
  - namespaceSelector: {}
  ports:
    - protocol: UDP
      port: 53
    - protocol: TCP
      port: 53
```

Checkpoint: Network policies created (for external RDS/ElastiCache)

Step 6: Remove Access Keys

What we're doing: Removing any access key secrets from Kubernetes (no longer needed with IRSA).

Find access key secrets:

```
# Search for access key secrets
kubectl get secrets -n h-game | grep -i "aws\|access\|key"

# Check if any secrets contain access keys
kubectl get secrets -n h-game -o yaml | grep -i
"access.*key\|secret.*key"
```

Delete access key secrets (if found):

```
# If you find access key secrets, delete them
# kubectl delete secret aws-credentials -n h-game

# Verify secrets deleted
kubectl get secrets -n h-game
```

Verify Pods still work (using IRSA):

```
# Check External Secrets Operator still works
kubectl get externalsecret -n h-game

# Should show: SecretSynced (working with IRSA, not access keys)

# Check backend Pods still work
kubectl get pods -l app=backend -n h-game

# Should show: Running (using IRSA if needed)
```

Checkpoint - Access Keys Removed:

- Access key secrets identified
- Access keys deleted
- Pods still working (using IRSA)
- No access keys in cluster

Access keys eliminated!**Step 7: Verify & Test**

What we're doing: Testing IRSA and network policies end-to-end.

Test 1: Verify IRSA working

```
Verify IRSA working
kubectl get pod -n external-secrets-system -l
app.kubernetes.io/name=external-secrets

# Check Pod has AWS credentials (from IAM role)
kubectl exec -it $(kubectl get pod -n external-secrets-system -l
app.kubernetes.io/name=external-secrets -o
jsonpath='{.items[0].metadata.name}') -n external-secrets-system -- env
| grep AWS

# Should show: AWS_ROLE_ARN, AWS_WEB_IDENTITY_TOKEN_FILE
```

Test 2: Verify External Secrets still works

```
# Check ExternalSecret status
kubectl get externalsecret -n h-game

# Should show: SecretSynced

# Verify Kubernetes Secret exists
kubectl get secret postgres-secret -n h-game

# Should show: Secret exists and synced
```

Test 3: Verify network policies (Pod to Pod)

```
# Test frontend can reach backend
kubectl exec -it $(kubectl get pod -l app=frontend -n h-game -o
jsonpath='{.items[0].metadata.name}') -n h-game -- nc -zv
backend-service 3001

# Expected: Connection successful
```

Test 4: Verify least privilege

```
Verify backend can reach RDS (external - network policy allows this)
kubectl exec -it $(kubectl get pod -l app=backend -n h-game -o
jsonpath='{.items[0].metadata.name}') -n h-game -- nc -zv
h-game-postgres.cyb44k06izcw.us-east-1.rds.amazonaws.com 5432

# Expected: Connection successful (RDS is external, network policy
allows egress)
```

Test 5: Verify least privilege

```
aws iam get-role-policy \
  --role-name $(terraform output -raw eks_cluster_name | sed
's/h-game-prod/h-game-learning/')-external-secrets-role \
  --policy-name secrets-manager-access

# Should show: Only Secrets Manager read access
```

Checkpoint: IRSA and network policies verified

Architecture Notes

Your Current Setup:

- **Database:** RDS PostgreSQL (external AWS service)
- **Cache:** ElastiCache Redis (external AWS service)
- **Backend:** Kubernetes Pod (connects to RDS/ElastiCache)
- **Frontend:** Kubernetes Pod (connects to Backend)

Network Policy Limitations:

- Network policies can restrict **Pod-to-Pod** traffic within cluster
- Network policies **cannot** effectively restrict traffic to **external services** (RDS, ElastiCache which we have)
- For external services, use **Security Groups** (we already have this in Terraform) and **VPC endpoints**

Zero Trust for External Services:

- Security Groups (Terraform already configures these)
- IAM roles with least privilege (IRSA - this task)
- Network policies for Pod-to-Pod (this task)
- VPC endpoints (optional, for private connectivity)
- Service mesh (optional, for advanced zero-trust)

Quick Reference Checklist

```
# 1. Verify OIDC provider (already exists)
aws iam list-open-id-connect-providers

# 2. Create IAM roles (Terraform)
cd h-game-terraform
# Add irsa-external-secrets.tf and irsa-backend.tf
terraform apply

# 3. Create Service Accounts
kubectl apply -f external-secrets-sa.yaml
kubectl apply -f backend-sa.yaml

# 4. Update Deployments
```

```
kubectl edit deployment backend -n h-game # Add serviceAccountName
```

```
# 5. Create Network Policies
```

```
kubectl apply -f network-policy-backend.yaml
```

```
kubectl apply -f network-policy-frontend.yaml
```

```
# 6. Remove access keys
```

```
kubectl get secrets -n h-game | grep -i aws
```

```
# 7. Verify
```

```
kubectl get externalsecret -n h-game
```

```
kubectl get pods -l app=backend -n h-game
```


Step 8: Commit Your Work

What we're doing: Committing IRSA and network policy configuration to Git.

```
# Navigate to project root
cd ~/projects/H-Game

# Add Kubernetes manifests
git add k8s/network-policy-*.yaml
git add k8s/backend-sa.yaml

# Navigate to Terraform directory
cd ~/projects/h-game-terraform

# Add Terraform files
git add irsa-*.tf
git add oidc-provider.tf

# Commit with clear message
cd ~/projects/H-Game
git commit -m "feat(security): Implement IRSA and zero trust

- Configure IRSA for Pod authentication (no access keys)
- Create IAM roles with least-privilege policies
- Add network policies for zero-trust networking
- Remove access keys from cluster
- Implement least-privilege access

Closes: DEVOPS-6104"

# Push to remote
git push origin main
```

Checkpoint - Work Committed:

- Kubernetes manifests committed
- Terraform files committed
- Clear commit message
- Changes pushed to remote

Phase 3: Validation & Consolidation (20 min)

Quiz: IAM & Zero Trust Understanding

Answer these questions to verify your understanding:

Question 1: What is IRSA?

Answer:

- **IRSA** = IAM Roles for Service Accounts
- Allows Pods to assume IAM roles instead of using access keys
- Temporary credentials (expire after 1 hour)
- Different role per service (least privilege)

Key point: IRSA eliminates access keys entirely.

Question 2: Why use network policies?

Answer:

- **Restrict Pod-to-Pod traffic** - Only allow necessary connections
- **Zero trust** - Never trust, always verify
- **Limit damage** - If one Pod is compromised, damage is limited

Key point: Network policies implement zero-trust networking.

Question 3: What is least privilege?

Answer:

- **Grant minimum required permissions** - Only what's needed
- **One IAM role per service** - Different permissions per service
- **Regular audit** - Review and reduce permissions over time

Key point: Least privilege reduces security risk.

Knowledge Check

Test your understanding before moving on:

1. What is IRSA?

- ☐ a) A secret storage system
- ☐ b) IAM Roles for Service Accounts
- ☐ c) A Kubernetes network feature

2. Why use IRSA instead of access keys?

- ☐ a) Access keys are free
- ☐ b) IRSA provides temporary credentials, least privilege
- ☐ c) Access keys are more secure

3. What are network policies?

- ☐ a) IAM policies for networks
- ☐ b) Firewall rules for Pods
- ☐ c) Security group rules

4. What is least privilege?

- ☐ a) Grant all permissions
- ☐ b) Grant minimum required permissions
- ☐ c) Grant permissions to all services

5. Do network policies block all traffic by default?

- ☐ a) Yes, default deny
- ☐ b) No, default allow
- ☐ c) Depends on configuration

Score:

- 5/5: Excellent! Move to D5
- 3-4/5: Review weak areas
- <3/5: Re-read Phase 1

Checklist: Did You Complete Everything?

Before moving on, check these boxes:

Understanding Phase

- I understand what IRSA is
- I know why zero trust matters
- I understand least privilege
- I can explain network policies

Implementation Phase

- OIDC provider verified/created
- IAM roles created
- Service Accounts created with annotations
- Deployments updated to use Service Accounts
- Network policies created
- Access keys removed
- Workflow tested end-to-end

Verification Phase

- Can explain IRSA flow
- Can verify Pods using IAM roles
- Can verify network policies working
- Can verify least privilege

Git

- Committed Kubernetes manifests
- Committed Terraform files
- Documented IRSA setup

Key Takeaways

The Big Picture

What you learned in this task:

- IRSA configured (Pods use IAM roles)
- Access keys eliminated
- Network policies restrict traffic
- Least-privilege IAM policies
- Zero trust implemented

Infrastructure is now zero trust!

Critical Concepts

1. IRSA:

- IAM Roles for Service Accounts
- Pods use IAM roles (no access keys)
- Temporary credentials (expire after 1 hour)
- Different role per service

2. Least Privilege:

- Grant minimum required permissions
- One IAM role per service
- Regular audit of permissions

3. Network Policies:

- Firewall rules for Pods
- Default deny (block everything)
- Allow specific connections only

4. Zero Trust:

- Never trust, always verify
- Minimum required access
- Limit damage if compromised

What's Next?

Congratulations! You've implemented zero trust!

Next: TASK D5 - Compliance & Auditing

In D5, you'll learn:

- How to enable Security Hub
- How to analyze CloudTrail logs
- How to implement compliance checks
- How all security layers work together

But most importantly: You now understand zero trust. Combined with scanning and secrets management, you have complete security coverage!

Proceed to TASK D5: Compliance & Auditing

TASK D5: Compliance & Auditing

Time: 2 hours | **Priority:** High | **Ticket:** DEVOPS-6105 | **Cost:** Free

Mission Brief

Enable AWS Security Hub for centralized security findings, configure CloudTrail for audit logging, and set up compliance monitoring against CIS AWS Foundations Benchmark.

Outcomes:

- Understand compliance and auditing requirements
- Enable AWS Security Hub
- Configure CloudTrail audit logging
- Set up CIS AWS Foundations Benchmark
- Create compliance dashboards
- Understand automated remediation

Prerequisites: Complete D1-D4 (all previous security tasks)

Setup Note: Security Hub aggregates findings from multiple AWS services. CloudTrail logs all API calls.

Cost Note: This task costs \$0. Security Hub free tier: First 100 findings/month free. CloudTrail: First 90 days free, then ~\$2-5/month.

Compliance is automated security monitoring.

Just like a health check tells you how your body is doing, **AWS Security Hub** tells you how secure your AWS environment is continuously, not once in a while.

Instead of guessing whether you're secure or compliant, Security Hub **shows you the facts.**

Mental Model (Before vs After)**Before Compliance Monitoring**

- Security status is unknown
- Audits are manual and stressful
- Compliance is checked only when requested
- Issues are found late

After Compliance Monitoring

- Security is monitored continuously
- All findings are in one place
- Compliance checks run automatically
- Audit-ready reports are always available

Difference:

Not *hoping* you're compliant but, **knowing** you are.

Phase 1: Understanding Compliance & Auditing

What Is Compliance Monitoring?

Compliance monitoring means:

- Continuously checking your infrastructure against security standards
- Tracking security issues automatically
- Keeping an audit trail of all activity

The Problem (Without a Tool)

- Security issues are spread across many services
- No single view of your security posture
- Audits take weeks
- Compliance is reactive

The Solution: AWS Security Hub

AWS Security Hub provides:

- A centralized security dashboard
- Continuous monitoring
- Automated checks against standards like **CIS AWS Foundations**
- Aggregated findings from multiple AWS services

What Security Hub Monitors:

- Container vulnerabilities (from **ECR / Trivy**)
- Secrets rotation (from **AWS Secrets Manager**)
- IAM policy risks
- S3 misconfigurations
- Audit logs from **CloudTrail**

Key Insight:

Security Hub is the **single source of truth** for security and compliance.

Why Compliance Matters

Without Security Hub

- You deploy and hope security is correct
- When asked for an audit, you scramble
- Manual reviews take weeks
- Problems are found late

With Security Hub

- Security is monitored from day one
- Audit requests are answered in minutes
- Compliance status is always visible
- Reports are exportable anytime

Key Concepts You Must Understand

- **Security Hub:** Central dashboard for security findings
- **CloudTrail:** Logs all AWS API activity (who did what, when)
- **CIS Benchmark:** A set of AWS security best practices
- **Compliance:** Continuous verification, not a one-time check

If you can explain these in simple terms, you understand this section.

Phase 2: Implementation Overview

What Changes After This Task

Before

- No centralized visibility
- No compliance tracking
- Manual audits
- No audit trail

After

- Security Hub enabled
- CloudTrail logging active
- CIS benchmark checks running
- Compliance dashboard ready
- Audit-ready reports available

Step-by-Step Plan

1. Enable **CloudTrail** for audit logging
2. Enable **AWS Security Hub**
3. Turn on **CIS AWS Foundations Benchmark**
4. Configure security standards
5. Review and understand findings
6. Create a compliance dashboard

One Line Takeaway

Compliance is not a one-time audit it's continuous visibility into your security posture, automated by Security Hub.

Step 1: Enable CloudTrail

What we're doing: Configuring CloudTrail to log all AWS API calls for audit and compliance.

 **IMPORTANT:** CloudTrail already exists! Update it instead of creating new.

Check if CloudTrail exists:

```
# List existing CloudTrail trails
aws cloudtrail list-trails --region us-east-1

# Get trail name from Terraform
cd ~/projects/h-game-terraform
TRAIL_NAME=$(terraform output -raw cloudtrail_arn | cut -d '/' -f 2)
echo "Trail name: $TRAIL_NAME"

# Check trail status
aws cloudtrail get-trail-status --name $TRAIL_NAME --region us-east-1
```

Update CloudTrail in Terraform (add missing features):

```
# File: cloudtrail.tf (UPDATE EXISTING FILE)
# Add these improvements to your existing CloudTrail configuration

# Add S3 bucket public access block (MISSING in current config)
resource "aws_s3_bucket_public_access_block" "cloudtrail_logs" {
  bucket = aws_s3_bucket.cloudtrail_logs.id

  block_public_acls       = true
  block_public_policy     = true
  ignore_public_acls     = true
  restrict_public_buckets = true
}

# Update CloudTrail trail (ADD is_multi_region_trail)
resource "aws_cloudtrail" "main" {
  name           = "${local.name_prefix}-cloudtrail"
  s3_bucket_name = aws_s3_bucket.cloudtrail_logs.id
```

```

include_global_service_events = true
is_multi_region_trail         = true # ← ADD THIS: Log all
regions
enable_logging                = true
enable_log_file_validation    = true

event_selector {
  read_write_type              = "All"
  include_management_events    = true
  exclude_management_event_sources = []

  # Optional: Add data resource selector for S3 (more specific
  logging)
  data_resource {
    type   = "AWS::S3::Object"
    values = ["arn:aws:s3:::*/*"]
  }
}

tags = merge(
  local.common_tags,
  {
    Name = "${local.name_prefix}-cloudtrail"
    Task = "D5" # Update task tag
  }
)

depends_on = [aws_s3_bucket_policy.cloudtrail_logs]
}

# Update S3 bucket policy (ADD SourceArn condition for better
security)
resource "aws_s3_bucket_policy" "cloudtrail_logs" {
  bucket = aws_s3_bucket.cloudtrail_logs.id

  policy = jsonencode({
    Version = "2012-10-17"

```

```

Statement = [
  {
    Sid      = "AWSCloudTrailAclCheck"
    Effect   = "Allow"
    Principal = {
      Service = "cloudtrail.amazonaws.com"
    }
    Action    = "s3:GetBucketAcl"
    Resource  = aws_s3_bucket.cloudtrail_logs.arn
    Condition = {
      StringEquals = {
        "AWS:SourceArn" =
"arn:aws:cloudtrail:${var.aws_region}:${data.aws_caller_identity.current.account_id}:trail/${local.name_prefix}-cloudtrail" # ← ADD THIS
      }
    }
  },
  {
    Sid      = "AWSCloudTrailWrite"
    Effect   = "Allow"
    Principal = {
      Service = "cloudtrail.amazonaws.com"
    }
    Action    = "s3:PutObject"
    Resource  = "${aws_s3_bucket.cloudtrail_logs.arn}/*"
    Condition = {
      StringEquals = {
        "s3:x-amz-acl"      = "bucket-owner-full-control"
        "AWS:SourceArn" =
"arn:aws:cloudtrail:${var.aws_region}:${data.aws_caller_identity.current.account_id}:trail/${local.name_prefix}-cloudtrail" # ← ADD THIS
      }
    }
  }
]
}))
}

```


Apply Terraform updates:

```
cd ~/projects/h-game-terraform
terraform plan
# Should show:
# ~ aws_cloudtrail.main (update in place)
# + aws_s3_bucket_public_access_block.cloudtrail_logs (new)
# ~ aws_s3_bucket_policy.cloudtrail_logs (update in place)

terraform apply
# Type 'yes' when prompted
```

Verify CloudTrail is logging:

```
# Get trail name
TRAIL_NAME=$(terraform output -raw cloudtrail_arn | cut -d '/' -f 2)

# Check CloudTrail status
aws cloudtrail get-trail-status --name $TRAIL_NAME --region us-east-1

# Expected output:
# {
#   "IsLogging": true,
#   "LatestCloudWatchLogsDeliveryTime": "2024-01-15T10:30:00Z",
#   ...
# }

# Check recent events (wait a few minutes after creating/updating
# trail)
aws cloudtrail lookup-events --max-results 5 --region us-east-1
```

Checkpoint: CloudTrail updated with multi-region logging and improved security

Step 2: Enable Security Hub

What we're doing: Enabling AWS Security Hub for centralized security findings.

Enable Security Hub:

```
# Enable Security Hub
aws securityhub enable-security-hub --region us-east-1

# Expected output:
# (no output on success)

# Verify Security Hub is enabled
aws securityhub describe-hub --region us-east-1

# Expected output:
# {
#   "HubArn":
#     "arn:aws:securityhub:us-east-1:123456789012:hub/default",
#   "SubscribedAt": "2024-01-15T10:30:00Z",
#   ...
# }
```

Or enable in Terraform Recommended:

```
# Add Security Hub configuration
cat > security-hub.tf << 'EOF'
# File: security-hub.tf (NEW FILE)
# =====
# AWS SECURITY HUB
# =====
# Centralized security findings dashboard

resource "aws_securityhub_account" "main" {
  enable_default_standards = false # We'll enable standards
  separately
}

# Enable CIS AWS Foundations Benchmark
resource "aws_securityhub_standards_subscription" "cis" {
  standards_arn =
    "arn:aws:securityhub:${var.aws_region}::standards/cis-aws-foundations-
    benchmark/v/1.2.0" # ← FIXED: Correct ARN format

  depends_on = [aws_securityhub_account.main]
}

# Optional: Enable AWS Foundational Security Best Practices
resource "aws_securityhub_standards_subscription" "aws_foundational"
{
  standards_arn =
    "arn:aws:securityhub:${var.aws_region}::standards/aws-foundational-se-
    curity-best-practices/v/1.0.0"

  depends_on = [aws_securityhub_account.main]
}

output "security_hub_arn" {
  description = "ARN of Security Hub"
  value       = aws_securityhub_account.main.id
}
```

```
cd ~/projects/h-game-terraform
terraform plan
# Should show:
# + aws_securityhub_account.main
# + aws_securityhub_standards_subscription.cis
# + aws_securityhub_standards_subscription.aws_foundational

terraform apply
# Type 'yes' when prompted
```

Verify Security Hub enabled:

```
# Check Security Hub status
aws securityhub describe-hub --region us-east-1

# List enabled standards
aws securityhub get-enabled-standards --region us-east-1

# Expected: Shows CIS benchmark if enabled
```

Checkpoint - Security Hub Enabled CIS benchmark

Step 3: Enable CIS Benchmark

What we're doing: Enabling CIS AWS Foundations Benchmark for compliance monitoring.

Enable CIS benchmark:

```
# Get CIS benchmark ARN
CIS_ARN="arn:aws:securityhub::ruleset/cis-aws-foundations-benchmark/
v/1.2.0"

# Subscribe to CIS benchmark
aws securityhub batch-enable-standards \
  --standards-subscription-requests StandardsArn=$CIS_ARN \
```

```

--region us-east-1

# Expected output:
# {
#   "StandardsSubscriptions": [
#     {
#       "StandardsSubscriptionArn":
#       "arn:aws:securityhub:us-east-1:123456789012:subscription/cis-aws-foun
#       dations-benchmark/v/1.2.0/...",
#       ...
#     }
#   ]
# }

```

Or use Terraform (from Step 2):

```

# If you used Terraform in Step 2, CIS benchmark is already enabled
# Verify:
terraform output security_hub_arn

```

Verify CIS benchmark enabled:

```

# List enabled standards
aws securityhub get-enabled-standards --region us-east-1

# Should show CIS benchmark in the list

# Get CIS benchmark status
aws securityhub get-findings \
  --filters
  '{"ComplianceStatus":[{"Value":"FAILED","Comparison":"EQUALS"}]}' \
  --region us-east-1 \
  --max-results 10

# Expected: Shows any failed compliance checks

```

Checkpoint - CIS Benchmark Enabled:

- CIS benchmark subscribed
- Compliance checks running
- Findings being generated

CIS benchmark enabled!**Step 4: Configure Security Standards**

What we're doing: Configuring additional security standards and enabling security checks.

Enable additional standards (optional):

```
# Enable AWS Foundational Security Best Practices
aws securityhub batch-enable-standards \
  --standards-subscription-requests
StandardsArn=arn:aws:securityhub:us-east-1::standards/aws-foundational-security-best-practices/v/1.0.0 \
  --region us-east-1

# Enable PCI-DSS (if needed for payment processing)
# aws securityhub batch-enable-standards \
#   --standards-subscription-requests
StandardsArn=arn:aws:securityhub:us-east-1::standards/pci-dss/v/3.2.1 \
#   --region us-east-1
```

Enable security checks:

```
# Security Hub automatically runs checks based on enabled standards
# No additional configuration needed

# Verify checks are running
aws securityhub get-findings \
  --filters
'{"RecordState":[{"Value":"ACTIVE","Comparison":"EQUALS"}]}' \
  --region us-east-1 \
  --max-results 10
```

```
# Expected: Shows active findings
```

Configure finding aggregation (if multi-region):

```
# If you have resources in multiple regions, configure finding
aggregation
# For single region, this is optional

# Create finding aggregator
aws securityhub create-finding-aggregator \
  --region us-east-1 \
  --region-linking-mode ALL_REGIONS \
  --region us-east-1

# Expected output:
# {
#   "FindingAggregatorArn":
#     "arn:aws:securityhub:us-east-1:123456789012:aggregator/...",
#   ...
# }
```

Checkpoint - Standards Configured:

- CIS benchmark enabled
- Security checks running
- Findings being generated
- Ready for review

Standards configured!

Step 5: Review Findings

What we're doing: Analyzing security findings and understanding compliance status.

View all findings:

```
# Get all findings
aws securityhub get-findings \
  --region us-east-1 \
  --max-results 50 \
  --output json > security-findings.json

# Count findings by severity
aws securityhub get-findings \
  --region us-east-1 \
  --filters
'{"SeverityLabel":[{"Value":"CRITICAL","Comparison":"EQUALS"}]}' \
  --query 'Findings | length(@)'

# Expected: Number of critical findings
```

View findings by compliance status:

```
# Get failed compliance checks
aws securityhub get-findings \
  --filters
'{"ComplianceStatus":[{"Value":"FAILED","Comparison":"EQUALS"}]}' \
  --region us-east-1 \
  --max-results 10 \
  --output table

# Expected: Shows failed compliance checks with details
```

View findings by resource type:

```
# Get findings for S3 buckets
aws securityhub get-findings \
  --filters
'{"ResourceType":[{"Value":"AwsS3Bucket","Comparison":"EQUALS"}]}' \
  --region us-east-1 \
  --max-results 10

# Get findings for IAM
aws securityhub get-findings \
  --filters
'{"ResourceType":[{"Value":"AwsIamRole","Comparison":"EQUALS"}]}' \
  --region us-east-1 \
  --max-results 10
```

View compliance summary:

```
# Get compliance summary
aws securityhub get-findings \
  --filters
'{"ComplianceStatus":[{"Value":"FAILED","Comparison":"EQUALS"}]}' \
  --region us-east-1 \
  --query 'Findings[*].Compliance.Status' \
  --output text | sort | uniq -c

# Expected: Count of findings by compliance status
```

Checkpoint - Findings Reviewed:

- Findings retrieved
- Compliance status checked
- Issues identified
- Ready for remediation

Findings reviewed!

Step 6: Create Compliance Dashboard

What we're doing: Creating a simple compliance dashboard using AWS CLI and CloudWatch (or viewing in AWS Console).

View Security Hub dashboard in AWS Console:

```
# Open Security Hub in browser
echo "Open Security Hub dashboard:"
echo
"https://console.aws.amazon.com/securityhub/home?region=us-east-1"

# Or use AWS CLI to get summary
aws securityhub get-findings \
  --region us-east-1 \
  --filters
'{"RecordState":[{"Value":"ACTIVE","Comparison":"EQUALS"}]}' \
  --query
'Findings[*].{Severity:Severity.Label,Title:Title,Compliance:Compliance.Status}' \
  --output table
```

Create compliance summary script:

```
# Navigate to project directory
cd ~/projects/H-Game
mkdir -p scripts

# Create compliance summary script
cat > scripts/compliance-summary.sh << 'EOF'
#!/bin/bash
# =====
# COMPLIANCE SUMMARY SCRIPT
# =====
# Generates compliance summary from Security Hub

REGION=${1:-us-east-1}

echo "Security Hub Compliance Summary"
echo "=====
echo ""
```

```

# Count findings by severity
echo "Findings by Severity:"
echo "-----"
CRITICAL=$(aws securityhub get-findings \
  --region $REGION \
  --filters '{"SeverityLabel":[{"Value":"CRITICAL","Comparison":"EQUALS"}]}' \
  --query 'Findings | length(@)' \
  --output text 2>/dev/null || echo "0")
echo "  CRITICAL: $CRITICAL"

HIGH=$(aws securityhub get-findings \
  --region $REGION \
  --filters '{"SeverityLabel":[{"Value":"HIGH","Comparison":"EQUALS"}]}' \
  --query 'Findings | length(@)' \
  --output text 2>/dev/null || echo "0")
echo "  HIGH: $HIGH"

MEDIUM=$(aws securityhub get-findings \
  --region $REGION \
  --filters '{"SeverityLabel":[{"Value":"MEDIUM","Comparison":"EQUALS"}]}' \
  --query 'Findings | length(@)' \
  --output text 2>/dev/null || echo "0")
echo "  MEDIUM: $MEDIUM"

LOW=$(aws securityhub get-findings \
  --region $REGION \
  --filters '{"SeverityLabel":[{"Value":"LOW","Comparison":"EQUALS"}]}' \
  --query 'Findings | length(@)' \
  --output text 2>/dev/null || echo "0")
echo "  LOW: $LOW"

echo ""
echo "Compliance Status:"
echo "-----"
PASSED=$(aws securityhub get-findings \
  --region $REGION \
  --filters '{"ComplianceStatus":[{"Value":"PASSED","Comparison":"EQUALS"}]}' \
  --query 'Findings | length(@)' \
  --output text 2>/dev/null || echo "0")
echo "  PASSED: $PASSED"

FAILED=$(aws securityhub get-findings \
  --region $REGION \

```

```

--filters '{"ComplianceStatus":[{"Value":"FAILED","Comparison":"EQUALS"}]}' \
--query 'Findings | length(@)' \
--output text 2>/dev/null || echo "0")
echo "  FAILED: $FAILED"

WARNING=$(aws securityhub get-findings \
  --region $REGION \
  --filters '{"ComplianceStatus":[{"Value":"WARNING","Comparison":"EQUALS"}]}' \
  \
  --query 'Findings | length(@)' \
  --output text 2>/dev/null || echo "0")
echo "  WARNING: $WARNING"

echo ""
echo "Top Failed Checks:"
echo "-----"
aws securityhub get-findings \
  --region $REGION \
  --filters '{"ComplianceStatus":[{"Value":"FAILED","Comparison":"EQUALS"}]}' \
  --query 'Findings[*].{Title:Title,Severity:Severity.Label}' \
  --output table \
  --max-results 10

echo ""
echo " View full dashboard:"
echo "https://console.aws.amazon.com/securityhub/home?region=$REGION"
EOF

# Make script executable
chmod +x scripts/compliance-summary.sh

# Run compliance summary
./scripts/compliance-summary.sh us-east-1

```

Create CloudWatch dashboard (optional):

```

# Create CloudWatch dashboard for Security Hub metrics
aws cloudwatch put-dashboard \
  --dashboard-name SecurityHub-Compliance \
  --dashboard-body file://dashboard.json \
  --region us-east-1

```

```
# Note: dashboard.json would contain CloudWatch dashboard
configuration
# For simplicity, use AWS Console or the compliance summary script
above
```

Checkpoint - Dashboard Created:

- Compliance summary script created
- Summary generated
- Findings visible
- Ready for reporting

Compliance dashboard ready!

Step 7: Verify & Test

What we're doing: Testing the complete compliance and auditing workflow.

Test 1: Verify CloudTrail logging

```
# Make a test API call
aws s3 ls

# Get trail name from Terraform
cd ~/projects/h-game-terraform
TRAIL_NAME=$(terraform output -raw cloudtrail_arn | cut -d '/' -f 2)

# Check CloudTrail for the event (wait a few minutes)
aws cloudtrail lookup-events \
  --lookup-attributes
  AttributeKey=EventName,AttributeValue=ListBuckets \
  --max-results 1 \
  --region us-east-1

# Expected: Shows the ListBuckets event
```

Test 2: Verify Security Hub findings

```
# Check if findings are being generated (may take 10-15 minutes)
aws securityhub get-findings \
  --region us-east-1 \
  --max-results 5 \
  --query 'Findings[*].{Title:Title,Severity:Severity.Label}' \
  --output table

# Expected: Shows recent findings (may be empty initially)
```

Test 3: Verify compliance status

```
# Run compliance summary
cd ~/projects/H-Game
./scripts/compliance-summary.sh us-east-1
```

```
# Expected: Shows compliance summary with counts
```

Test 4: Verify CIS benchmark

```
# Check CIS benchmark status
aws securityhub get-enabled-standards \
  --region us-east-1 \
  --query 'StandardsSubscriptions[?contains(StandardsArn,`cis`)].{ARN:StandardsArn,Status:StandardsStatus}' \
  --output table

# Expected: Shows CIS benchmark enabled
```

Checkpoint - Complete Workflow Verified:

Step 8: Commit Your Work

What we're doing: Committing compliance and auditing configuration to Git.

```
# Navigate to project root
cd ~/projects/H-Game

# Add compliance script
git add scripts/compliance-summary.sh

# Navigate to Terraform directory
cd ~/projects/h-game-terraform

# Add Terraform files
git add cloudtrail.tf
git add security-hub.tf

# Commit with clear message
cd ~/projects/H-Game
git commit -m "feat(security): Add compliance and auditing"
```

- Enable CloudTrail for audit logging
- Enable Security Hub for centralized findings
- Configure CIS AWS Foundations Benchmark
- Create compliance summary script
- Enable continuous compliance monitoring

Closes: DEVOPS-6105"

```
# Push to remote  
git push origin main
```

Checkpoint - Work Committed:

- Compliance script committed
- Terraform files committed
- Clear commit message
- Changes pushed to remote

Work committed!

Knowledge Check

Test your understanding before moving on:

1. **What is Security Hub?**

- ☐ a) A security tool
- ☐ b) Centralized security dashboard
- ☐ c) A compliance certification

2. **What does CloudTrail log?**

- ☐ a) Application logs
- ☐ b) All AWS API calls
- ☐ c) Security findings only

3. **What is CIS benchmark?**

- ☐ a) A security tool
- ☐ b) Security best practices for AWS
- ☐ c) A compliance certification

4. **Why enable compliance monitoring?**

- ☐ a) It's required by law
- ☐ b) Continuous security visibility and audit-ready reports
- ☐ c) It's free

5. **Is Security Hub free?**

- ☐ a) Yes, always
- ☐ b) First 100 findings/month free
- ☐ c) No, it costs money

Score:

- 5/5: Excellent! Section 6 complete!
- 3-4/5: Review weak areas
- <3/5: Re-read Phase 1

Critical Concepts

1. Compliance Monitoring:

- Security Hub: Centralized security dashboard
- CIS benchmark: Security best practices
- Continuous monitoring: Automated checks
- Audit-ready: Reports anytime

2. Audit Logging:

- CloudTrail: Logs all AWS API calls
- S3 storage: Encrypted, versioned
- Compliance requirement: Audit trail
- Security investigations: Track changes

3. Security Standards:

- CIS AWS Foundations Benchmark
- AWS Foundational Security Best Practices
- PCI-DSS (if needed)
- Custom standards

4. Automation:

- Security Hub runs checks automatically
- Findings aggregated automatically
- Compliance status tracked continuously
- No manual audits needed

What's Next?

Congratulations! You've completed Section 6 - DevSecOps!

Section 6 DevSecOps Complete!